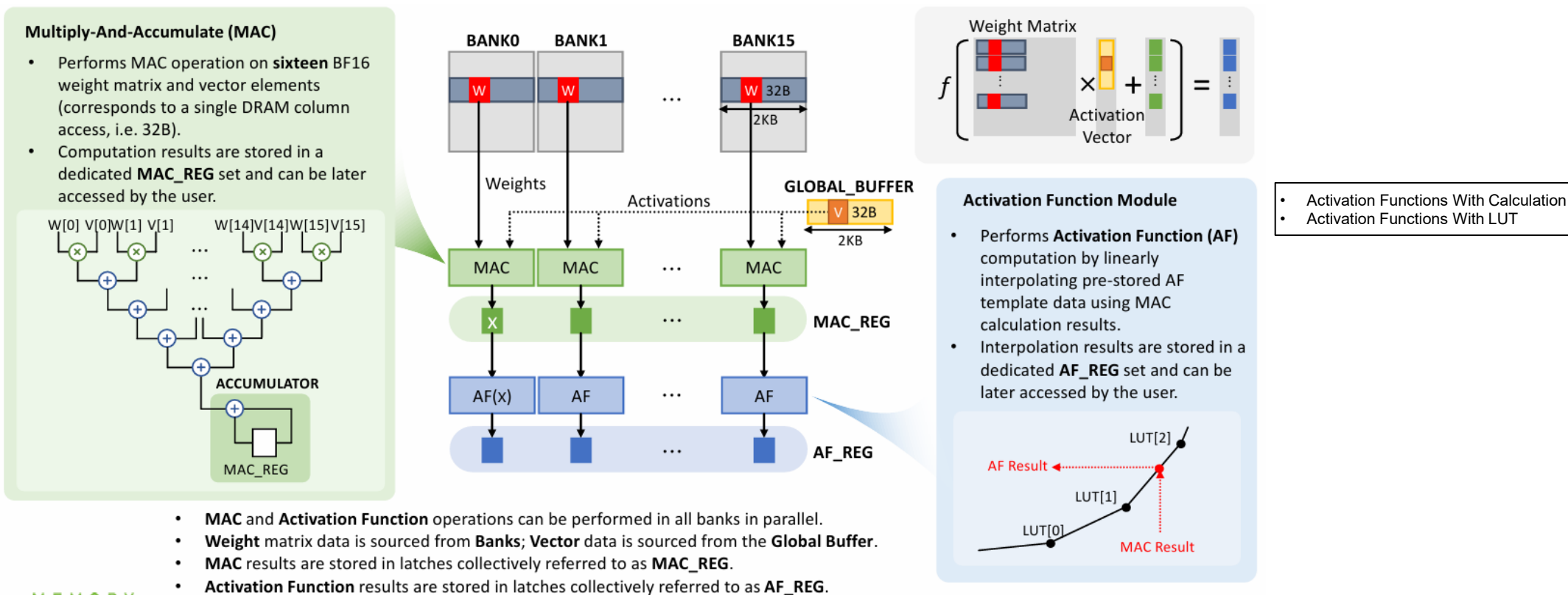

AiM Simulator Summary

***Intelligent System
Laboratory***

AiM Hardware

□ AiM Chip

- Components : Bank, MAC/AF accumulators(16bit each), Global Buffer(2kB)
- Data Burst → 16bit x 16
- 16개의 MAC unit이 병렬로 16burst동안 연산 → 16bit MAC x 16(=MAC units) x 16(=Burst Length)

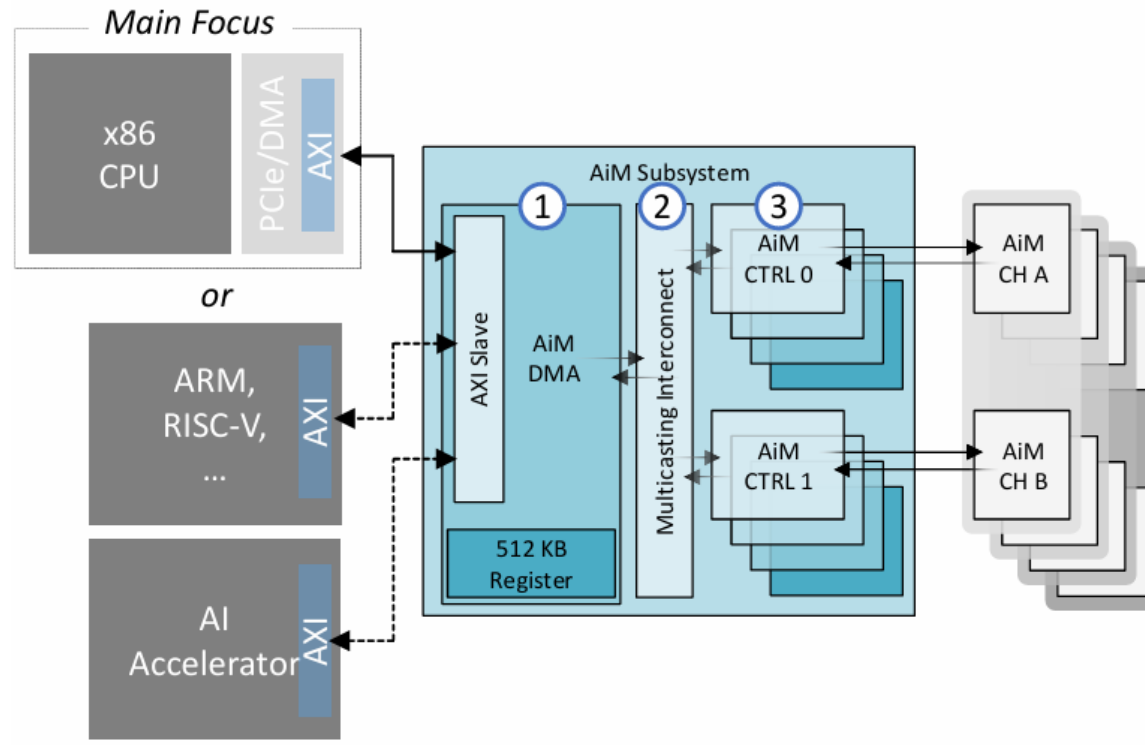


AiM Hardware

□ AiM Subsystem

- Components : AiM DMA, Memory Controllers(x32 in AiMSim)

AiM Subsystem is a hardware bridge between the host and the AiM devices. It is designed to 1) **maximize compute throughput** for a set of AiM devices and to 2) **minimize software stack overhead**.

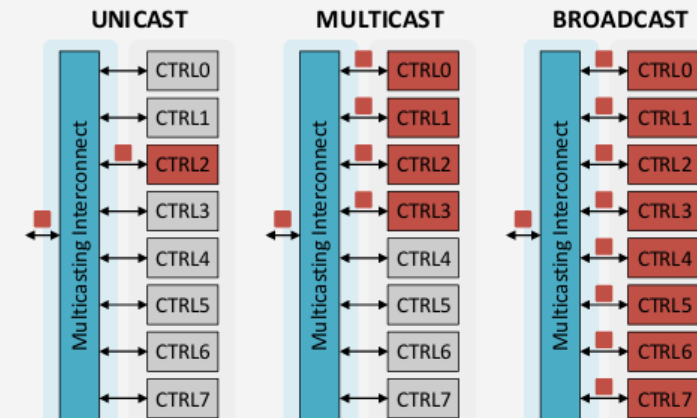


① AiM Command/Data DMA

Decodes AiM instructions from software and provides direct memory access for the host.

② AiM Multicasting Interconnect

Enables efficient workload distribution through flexible instruction parallelism. Supports unicast, multicast, and broadcast modes.



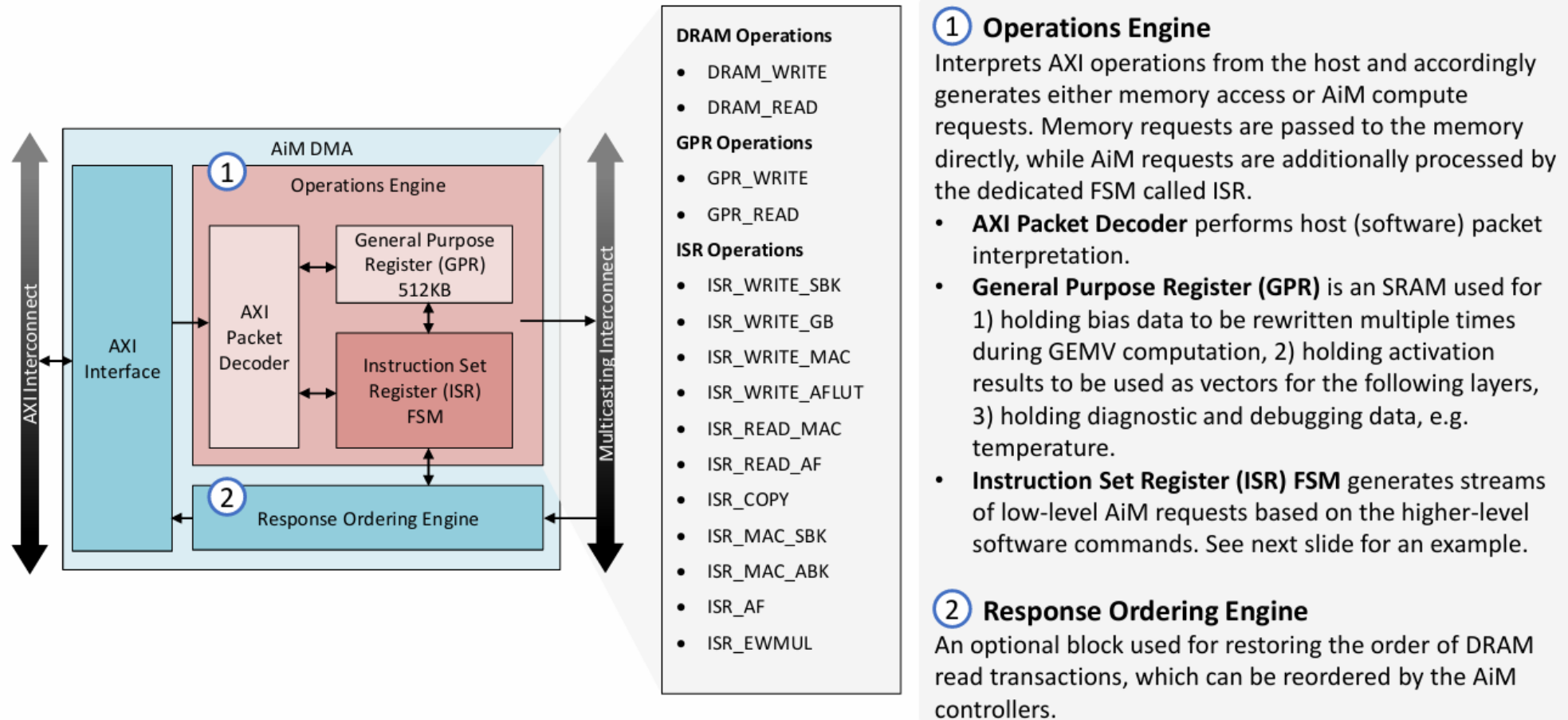
③ AiM Controller

Generates and schedules low-level AiM and typical DRAM commands.

AiM Hardware

□ AiM DMA

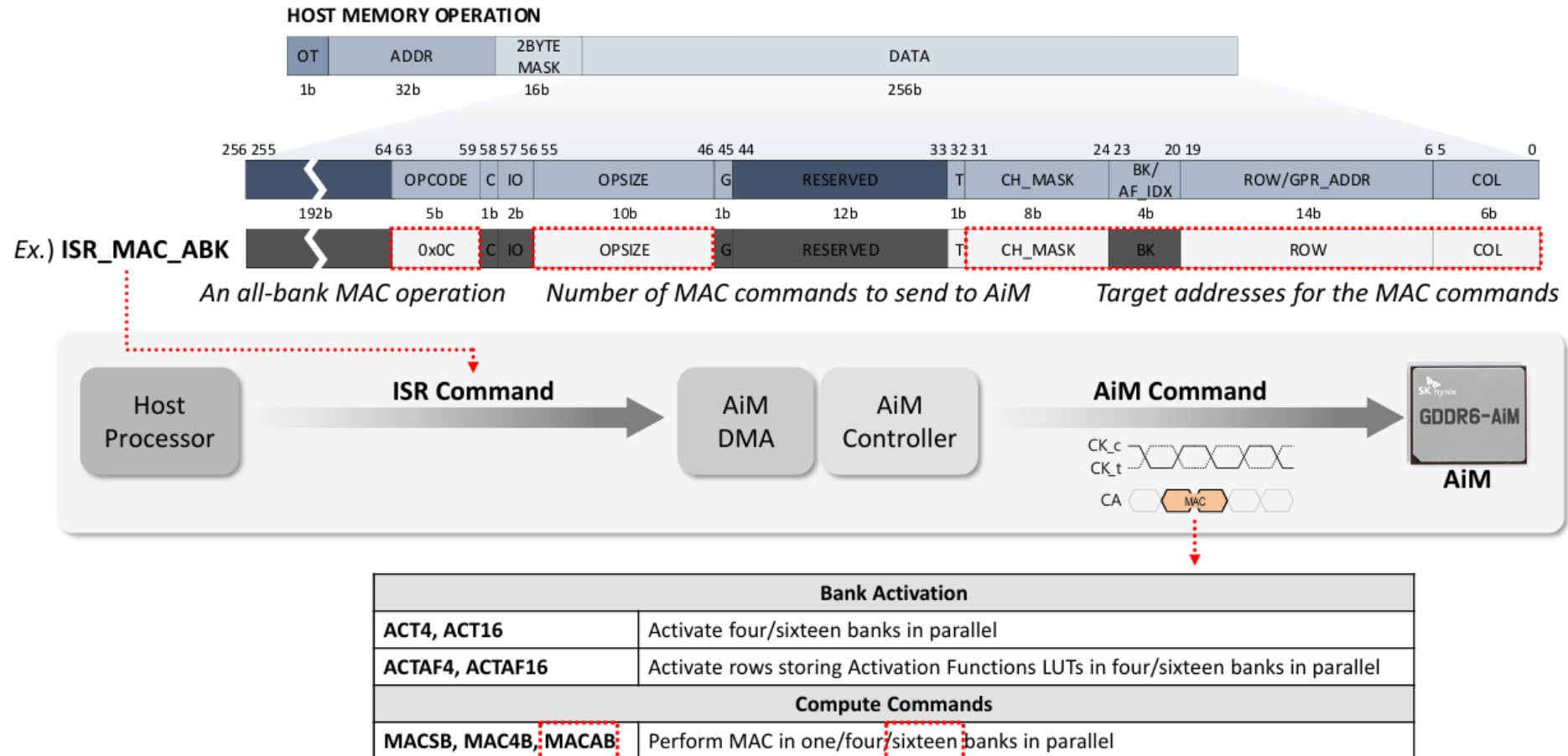
- Components : GPR(512kB) → 내부적으로 16,384개의 독립된 256bit entry으로 구성



AiM Hardware

❑ AiM ISR

- ISRCommand Format

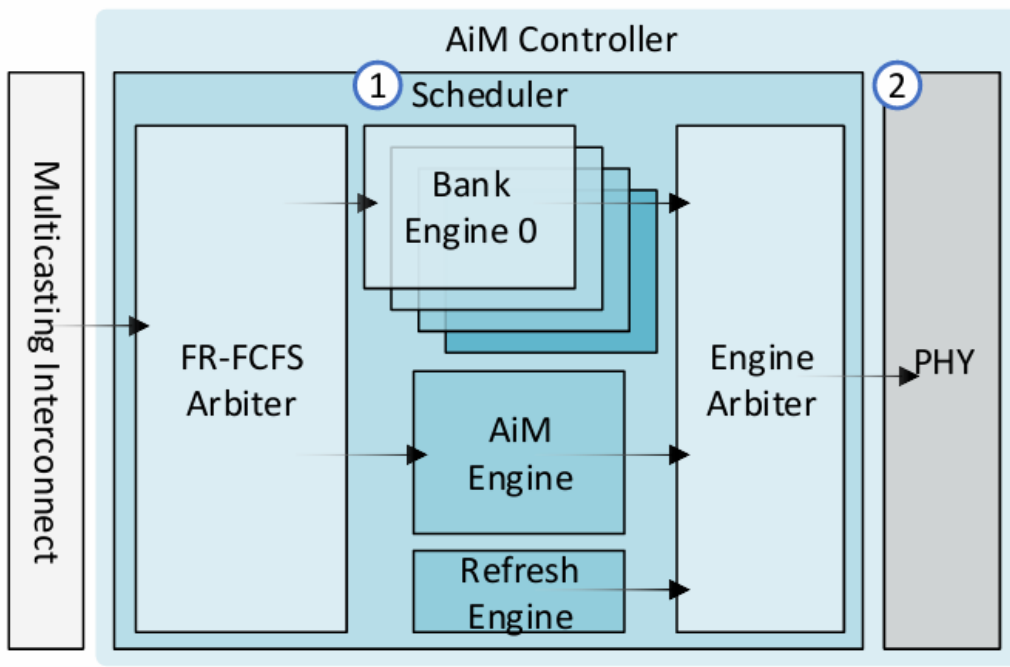


AiM Hardware

❑ AiM Controller

- AiM Controller

GDDR6-AiM Controller = GDDR6 Controller + AiM Command Scheduling



① Scheduler

- **FR-FCFS Arbiter** rearranges memory requests to reduce the number of row activations. AiM requests are always scheduled in-order.
- **Bank Engines** and **AiM Engine** generate and schedule DRAM commands (ACT, PREPB, MAC, ...) from the memory requests.
- **Refresh Engine** periodically generates and schedules refresh commands.
- **Engine Arbiter** multiplexes between the engines and issues prioritized DRAM commands to PHY.

② PHY

A standard GDDR6 DRAM PHY block. Details depend on the selected implementation technology (e.g. FPGA).

AiM Simulator CMDs

❑ Deep Learning CMDs

- Simulator의 CMDs

TABLE I NEWLY DEFINED DEDICATED DL CMD SET		
Type	CMD	Description
Bank Activation	ACT4, ACT16	Activate four/sixteen banks in parallel
	ACTAF4, ACTAF16	Activate rows storing activation function LUTs in four/sixteen banks parallel
Compute	MACSB, MAC4B, MACAB	Perform MAC in one/four/sixteen banks in parallel
	AF	Compute activation function in all banks
	EWML	Perform element-wise multiplication
Data	RDCP	Copy data from a bank to global buffer
	WRCP	Copy data from global buffer to a bank
	WRGB	Write to global buffer
	RDMAC	Read from MAC result register
	RDAF	Read from activation function result register
	WRBIAS	Write bias to MAC result register
	WRBK	Write to all activated banks in parallel

[GDDR6 based AiM paper의 Table]

유형	명령어	세부 사항
AiM	WR_SBK [GPR_0] [channel_mask] [bank] [row]	GPR에서 256b를 Single Bank로 Write
AiM	WR_ABK [GPR_0] [channel_mask] [row]	GPR에서 256b를 16개 Bank로 Write
AiM	WR_GB [opsize] [GPR_0] [channel_mask]	GPR에서 256b를 GB로 Write
AiM	WR_BIAS [GPR_0] [channel_mask]	GPR에서 256b를 16개 Bank의 16b MAC Reg로 Write
AiM	RD_MAC/RD_AF [GPR_0] [channel_mask]	16개 Bank의 16b MAC/AF Reg에서 256b를 GPR로 Read
AiM	RD_SBK [GPR_0] [channel_mask] [bank] [row]	Single Bank에서 256b를 GPR로 Write
AiM	COPY_BKGB [opsize] [channel_mask] [bank] [row]	16개 Bank에서 256b를 GB로 Copy
AiM	COPY_GBBK [opsize] [channel_mask] [bank] [row]	GB에서 256b를 16개 Bank로 Copy
AiM	MAC_SBK [opsize] [channel_mask] [bank] [row]	Single Bank에서 256b MAC 수행
AiM	MAC_ABK [opsize] [channel_mask] [row]	모든 Bank에서 256b MAC 수행

AiM Simulator CMDs

❑ Deep Learning CMDs

- Simulator의 CMDs

TABLE I NEWLY DEFINED DEDICATED DL CMD SET		
Type	CMD	Description
Bank Activation	ACT4, ACT16	Activate four/sixteen banks in parallel
	ACTAF4, ACTAF16	Activate rows storing activation function LUTs in four/sixteen banks parallel
Compute	MACSB, MAC4B, MACAB	Perform MAC in one/four/sixteen banks in parallel
	AF	Compute activation function in all banks
	EWMUL	Perform element-wise multiplication
Data	RDCP	Copy data from a bank to global buffer
	WRCP	Copy data from global buffer to a bank
	WRGB	Write to global buffer
	RDMAC	Read from MAC result register
	RDAF	Read from activation function result register
	WRBIAS	Write bias to MAC result register
	WRBK	Write to all activated banks in parallel

[GDDR6 based AiM paper의 Table]

유형	명령어	세부 사항
AiM	AF [channel_mask]	모든 Bank에서 Activation Func 수행
AiM	EWMUL [opsize] [channel_mask] [row]	4개 Bank에서 Elementwise Mult 수행
AiM	EWADD [opsize] [GPR_0] [GPR_1]	GPR을 사용한 Elementwise Add 수행
AiM	SYNC	ISR 명령어 간의 Barrier
AiM	EOC	계산 종료, 시뮬레이션 종료
W	MEM [channel_id] [bank] [row]	Bank에 대한 기존 Write 접근
R	MEM [channel_id] [bank] [row]	Bank로부터의 기존 Read 접근
W	GPR [GPR_0]	GPR에 256b Write
R	GPR [GPR_0]	GPR로부터 256b Read
W	CFR [CFR_0] [data]	Data를 이용해 CFR 구성

AiM Simulator Example Trace

□ 기본으로 주어진 all_isr.trace를 분석

```
1. # AiM [opcode] [opsize] [GPR_id_0] [GPR_id_1] [channel_mask] [bank_id] [row_id]
2. # R/W GPR [GPR_id]
3. # R/W CFR "[CFR_id {0: broadcast 1: EWUL_bg 2: afm}]" [data]
4. # R/W MEM [channel_id] [bank_id] [row_id]
5. # Access granularity [GPR: BURST bytes] [CFR: 2 bytes] [MEM: Burst bytes] where BURST = 32 bytes
6.
7. # Regular write to row [8-11] in bank [2] of channels [0-3]
8. # R/W MEM [channel_id] [bank_id] [row_id]
9. W MEM 0 2 8
10. W MEM 1 2 9
11. W MEM 2 2 10
12. W MEM 3 2 11
13.
14. # Writing to GPR [0-1] for bank data
15. # R/W GPR [GPR_id]
16. W GPR 0
17. W GPR 1
18.
19. # Writing to GPR [2-3] for global buffer data
20. # R/W GPR [GPR_id]
21. W GPR 2
22. W GPR 3
23.
24. # Writing to GPR [4-7] for bias data
25. # R/W GPR [GPR_id]
26. W GPR 4
27. W GPR 5
28. W GPR 6
29. W GPR 7
30.
31. # "Write [2-data = 512 bits] from GPR[0 1] to channels [0-3] bank [0] row [0] (channel mask must only specify 1 channel)"
32. # AiM [opcode] [GPR_id_0] [channel_mask] [bank_id] [row_id]
33. AiM WR_SBK 0 1 0 0
34. AiM WR_SBK 0 2 0 0
35. AiM WR_SBK 0 4 0 0
36. AiM WR_SBK 0 8 0 0
37. AiM WR_SBK 1 1 0 0
38. AiM WR_SBK 1 2 0 0
39. AiM WR_SBK 1 4 0 0
40. AiM WR_SBK 1 8 0 0
41.
42. # "Write [2-data = 512 bits] from GPR[2 3] to channels [0-3] global buffers"
43. # AiM [opcode] [opsize] [GPR_id_0] [channel_mask]
44. AiM WR_GB 2 2 15
45.
46. # Write [4-data = 1024 bits] from GPR[4-7] to channels [0-3] * 16 MAC accumulators
47. # AiM [opcode] [GPR_id_0] [channel_mask]
48. AiM WR_BIAS 4 15
49.
```

- Simul이기에, Data는 따로 주고 받지 않고, Address만 이용합니다.

1. **Memory Write (line 9-12)** - R/W MEM [channel_id] [bank_id] [row_id]

- W MEM 0 2 8 : Regular write to row[8] in bank[2] of channel[0]

2. **GPR Write (line 16-29)** - R/W GPR [GPR_id]

- W GPR 0 : Writing 256bit to GPR[0] (GPR: General Purpose Register)

3. **BANK Write (line 33-40)** - AiM [opcode] [GPR_id_0] [channel_mask] [bank_id] [row_id]

- AiM WR_SBK 0 1 0 0 : Write 256bit from GPR[0] to channel[0] bank[0] row[0]

4. **GB Write (line 44)** - AiM [opcode] [opsize] [GPR_id_0] [channel_mask]

- AiM WR_GB 2 2 15 :

Write [2-data = 512 bits] from GPR[2-3] to channels [0-3] global buffers

- opsize는 가져올 data 크기를 조절
- opsize = 2이면, 256b data 2개 (= 512b) 가져옴
- GPR은 1개 당 256b 저장 가능
- 512b 가져오려면 GPR id: 2부터 시작해 GPR id: 3까지 이용해야 함
- channel mask: 15 = 32'b0000_1111이므로 channel 0~3을 의미

4. **16개 BANK의 16b MAC Reg에 Write (line 48)** - AiM [opcode] [GPR_id_0] [channel_mask]

- AiM WR_BIAS 4 15 :

Write [4-data = 1024 bits] from GPR[4-7] to channels [0-3] * 16 MAC accumulators

AiM Simulator Example Trace

□ 기본으로 주어진 all_isr.trace를 분석

```
50. # set CFR[mr13.broadcast at address 0x0010] to 1 (MAC from global buffer)
51. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
52. W CFR 0 1
53.
54. # MAC operation between 2 columns of all banks in channels [0-3] with 2 columns of global buffer
55. # AiM [opcode] [opsize] [channel_mask] [row_id]
56. AiM MAC_ABK 2 15 0
57.
58. # set CFR[mr13.broadcast at address 0x0010] to 0 (MAC from next bank)
59. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
60. W CFR 0 0
61.
62. # MAC operation between 2 columns of half of the banks in channels [0-3] with 2 columns of the next banks
63. # AiM [opcode] [opsize] [channel_mask] [row_id]
64. AiM MAC_ABK 2 15 0
65.
66. # set CFR[mr13.broadcast at address 0x0010] to 1 (MAC from global buffer)
67. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
68. W CFR 0 1
69.
70. # MAC operation between 2 columns of bank [0] in channels [0-3] with 2 columns of global buffer
71. # AiM [opcode] [opsize] [channel_mask] [bank_id] [row_id]
72. AiM MAC_SBK 2 15 0 0
73.
74. # set CFR[mr13.broadcast at address 0x0010] to 0 (MAC from next bank)
75. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
76. W CFR 0 0
77.
78. # MAC operation between 2 columns of bank [0] in channels [0-3] with 2 columns of the next banks
79. # AiM [opcode] [opsize] [channel_mask] [bank_id] [row_id]
80. AiM MAC_SBK 2 15 0 0
81.
82. # set CFR[mr14.ewmul_bg at address 0x0018] to 1 (EWMUL only 1 bankgroup)
83. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
84. W CFR 1 1
85.
86. # "ELMUL operation between 1 bank group 2 columns of channels [0-3] with 2 columns of the next banks"
87. # AiM [opcode] [opsize] [channel_mask] [row_id]
88. AiM EWMUL 2 15 0
89.
90. # set CFR[mr14.ewmul_bg at address 0x0018] to 0 (EWMUL in all bankgroups)
91. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
92. W CFR 1 0
93.
94. # "ELMUL operation between all bank groups 2 columns of channels [0-3] with 2 columns of the next banks"
95. # AiM [opcode] [opsize] [channel_mask] [row_id]
96. AiM EWMUL 2 15 0
97.
98. # set CFR[mr13.afm at address 0x001C] to 1 (AF number 1)
99. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
100. W CFR 2 1
```

5. **set CFR (line 52)** - R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]

- W CFR 0 1 : set CFR[0] to 1 (= MAC from global buffer)

6. **MAC Operation (line 56)** - AiM [opcode] [opsize] [channel_mask] [row_id]

- AiM MAC_ABK 2 15 0 :

MAC : (2 columns of all banks in channels [0-3]) x (2 columns of global buffer)

7. **set CFR (line 60)** - R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]

- W CFR 0 0 : set CFR[0] to 0 (= MAC from next bank)

8. **MAC Operation (line 64)** - AiM [opcode] [opsize] [channel_mask] [row_id]

- AiM MAC_ABK 2 15 0 :

MAC : (2 columns of half banks in channels [0-3]) x (2 columns of the next banks)

... (생략) ...

9. **set CFR (line 84)** - R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]

- W CFR 1 1 : set CFR[1] to 1 (= EWMUL only 1 bankgroup)

10. **EWMUL Operation (line 88)** - AiM [opcode] [opsize] [channel_mask] [row_id]

- AiM EWMUL 2 15 0 :

(2 columns of 1 bank group in channels [0-3]) x (2 columns of next banks)

같은 bankgroup 내 -> Bank[i] × Bank[i+1] (simulation상에서 bankgroup id는 구현 X)

... (생략) ...

11. **set CFR (line 100)** - R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]

- W CFR 2 1 : set CFR[2] to 1 (= Activation Function 1, simul 상에서 Act Func들 정의는 X)

AiM Simulator Example Trace

□ 기본으로 주어진 all_isr.trace를 분석

```
101.
102. # AF operation in all banks of channels [0-3]
103. # AiM [opcode] [channel_mask]
104. AiM AF 15
105.
106. # Reading MAC accumulators of all banks of channels [0-3] to GPRs[8-11]
107. # AiM [opcode] [GPR_id_0] [channel_mask]
108. AiM RD_MAC 8 15
109.
110. # Reading AF results of all banks of channels [0-3] to GPRs[12-15]
111. # AiM [opcode] [GPR_id_0] [channel_mask]
112. AiM RD_AF 12 15
113.
114. # ELADD operation between GPRs[8-11] and GPRs[12-15]
115. # AiM [opcode] [opcode] [GPR_id_0] [GPR_id_1]
116. AiM EWADD 4 8 12
117.
118. # Copying 2 columns from bank 0 row 0 to global buffers in channels [0-3]
119. # AiM [opcode] [opcode] [channel_mask] [bank_id] [row_id]
120. AiM COPY_BKGB 2 15 0 0
121.
122. # Copying 2 columns from global buffers to bank 0 row 0 in channels [0-3]
123. # AiM [opcode] [opcode] [channel_mask] [bank_id] [row_id]
124. AiM COPY_GBBK 2 15 0 0
125.
126. # Read [2-data = 512 bits] from channels [0-3] bank [0] row [0] to GPR[20-27] (channel mask must only specify 1 channel)
127. # AiM [opcode] [GPR_id_0] [channel_mask] [bank_id] [row_id]
128. AiM RD_SBK 20 1 0 0
129. AiM RD_SBK 21 1 0 0
130. AiM RD_SBK 22 2 0 0
131. AiM RD_SBK 23 2 0 0
132. AiM RD_SBK 24 4 0 0
133. AiM RD_SBK 25 4 0 0
134. AiM RD_SBK 26 8 0 0
135. AiM RD_SBK 27 8 0 0
136.
137. # AiM [opcode] [GPR_id_0] [channel_mask] [row_id]
138. AiM WR_ABK 4 1 5
139. AiM WR_ABK 5 2 5
140. AiM WR_ABK 6 4 5
141. AiM WR_ABK 7 8 5
142.
143. # Finishing compute (MUST have this instruction at the end of the trace file)
144. # AiM [opcode]
145. AiM EOC
```

12. **AF Operation (line 104)** - AiM [opcode] [channel_mask]

- AiM AF 15:

모든 bank의 MAC accumulator 값에 AF를 적용 & AF regs에 저장

13. **Read MAC regs (line 108)** - AiM [opcode] [GPR_id_0] [channel_mask]

- AiM RD_MAC :

Read MAC accumulators of all banks in channel[0-3] to GPRs[8-11]

... (생략) ...

AiM Simulator Main Function

Main Function의 초기화 및 설정 부분

```
1. int main(int argc, char *argv[]) {
2.     Ramulator::AiMInfo::init();
3.
4.     // Parse command line arguments
5.     argparse::ArgumentParser program("Ramulator", "2.0");
6.     program.add_argument("-c", "--config").metavar("YAML configuration").help("String dump of the yaml configuration.");
7.     program.add_argument("-f", "--config_file").metavar("path-to-configuration-file").help("Path to a YAML configuration file.");
8.     program.add_argument("-p", "--param").metavar("KEY=VALUE").help("Specify parameter to override in the configuration file. Repeat this option to change multiple parameters.");
9.     program.add_argument("-t", "--trace_file").metavar("path-to-trace-file").help("Path to the trace file.");
10.
11.     try {
12.         program.parse_args(argc, argv);
13.     } catch (const std::runtime_error &err) {
14.         spdlog::error(err.what());
15.         std::cerr << program;
16.         std::exit(1);
17.     }
18.
19.     // Are we accepting the configuration YAML through commandline dump?
20.     bool use_dumped_yaml = false;
21.     std::string dumped_config;
22.     if (auto arg = program.present<std::string>("-c")) {
23.         use_dumped_yaml = true;
24.         dumped_config = *arg;
25.     }
26.
27.     // Are we getting a path to a YAML document?
28.     bool use_yaml_file = false;
29.     std::string config_file_path;
30.     if (auto arg = program.present<std::string>("-f")) {
31.         use_yaml_file = true;
32.         config_file_path = *arg;
33.     }
34.
35.     // Are we overriding some parameters in a YAML document from the command line?
36.     bool has_param_override = false;
37.     std::vector<std::string> params;
38.     if (auto arg = program.present<std::vector<std::string>>("-p")) {
39.         has_param_override = true;
40.         params = *arg;
41.     }
42.
43.     // Some sanity check of the inputs
44.     if (use_dumped_yaml && use_yaml_file) {
45.         spdlog::error("Dumped config and loaded config cannot be used together!");
46.         std::cerr << program;
47.         std::exit(1);
48.     } else if (!use_dumped_yaml && !use_yaml_file) {
49.         spdlog::error("No configuration specified!");
50.         std::cerr << program;
51.         std::exit(1);
52.     }
53.
54.     if (use_dumped_yaml && has_param_override) {
55.         spdlog::warn("Using dumped configuration. Parameter overrides with -p/--param will be ignored!");
56.     }
57.
58.     // Parse the configurations
59.     YAML::Node config;
60.     if (use_dumped_yaml) {
61.         std::string dumped_config = program.get<std::string>("-c");
62.         config = YAML::Load(dumped_config);
63.     } else if (use_yaml_file) {
64.         config = Ramulator::Config::parse_config_file(config_file_path, params);
65.     }
66.
67.     if (auto arg = program.present<std::string>("-t")) {
68.         Ramulator::trace_file_path = *arg;
69.         Ramulator::use_trace_file_path = true;
70.     }
71.
72.     // Instantiate the frontend of the simulated system, this is one of the top-level objects in Ramulator 2.0.
73.     // It also recursively instantiate all components in the frontend.
74.     auto frontend = Ramulator::Factory::create_frontend(config);
75.     // Instantiate the memory system of the simulated system, this is one of the top-level objects in Ramulator 2.0.
76.     // It also recursively instantiate all components in the memory system.
77.     auto memory_system = Ramulator::Factory::create_memory_system(config);
78.
79.     // Connect the frontend and the memory system together,
80.     // this recursively calls the "setup" function in all instantiated components
81.     // so that they can get each other's parameters (if needed) after their initialization
82.     frontend->connect_memory_system(memory_system);
83.     memory_system->connect_frontend(frontend);
84.
85.     // Get the relative clock ratio between the frontend and memory system
86.     int frontend_tick = frontend->get_clock_ratio();
87.     int mem_tick = memory_system->get_clock_ratio();
88.
89.     int tick_mult = frontend_tick * mem_tick;
```

1. 초기화 및 전역 상태 설정

- AiM ISR(Instruction Set Register) Info 초기화
- ISR 명령어 Mapping: String으로 들어오는 Trace Inst(ex. "WR_SBK" 등)를 Opcode enum으로 변환
- Type Mapping: Mem Access Region(GPR, CFR, MEM 등)의 정보를 Mapping

2. Command Line Parameters Parsing (ex. ./ramulator2 -f test/example.yaml -t test/all_isr.trace)

- -c / --config YAML 설정을 string으로 직접 전달 (dump 형식)
- -f / --config_file YAML 설정 파일 경로
- -p / --param 설정값 동적 override
- -t / --trace_file Instruction들이 포함된 Trace File 경로

3. 설정 File(ex. example.yaml) Parsing

4. Frontend Load 및 Memory System Load

- Yaml File에 주어진 Frontend Load 및 Memory System Load를 진행
 - ex. Frontend = AiMTrace / MemorySystem = AiMDRAM

request.h – init() 함수

```
210 public:
211     static void init() {
212         aim_opcode_to_str[Opcode::ISR_WB_SBK] = "ISR_WB_SBK";
213         opcode_str_to_aim_isr["ISR_WB_SBK"] = AIMISR(Opcode::ISR_WB_SBK,
214             (AIMISR::field::gpr_addr_0,
215             AIMISR::field::channel_mask,
216             AIMISR::field::bank_index,
217             AIMISR::field::row_addr),
218             false, // channel count req. num
219             false, // ADR_PRR_blocking
220             false, // require reg. fu. word
221             "column" // target_level
222         );
223
224         aim_opcode_to_str[Opcode::ISR_WB_AKB] = "ISR_WB_AKB";
225         opcode_str_to_aim_isr["ISR_WB_AKB"] = AIMISR(Opcode::ISR_WB_AKB,
226             (AIMISR::field::gpr_addr_0,
227             AIMISR::field::channel_mask,
228             AIMISR::field::row_addr),
```

AiM Simulator Main Function

□ Main Function의 Simulation 실행 부분

```
91. for (uint64_t i = 0;; i++) {
92.     if (((i % tick_mult) % mem_tick) == 0) {
93.         frontend->tick();
94.     }
95.
96.     if (frontend->is_finished()) {
97.         break;
98.     }
99.
100.    if ((i % tick_mult) % frontend_tick == 0) {
101.        memory_system->tick();
102.    }
103. }
104.
105. // Finalize the simulation. Recursively print all statistics from all components
106. frontend->finalize();
107. memory_system->finalize();
108.
109. return 0;
110. }
111.
```

5. Simulation 실행 Loop

- Ramulator2.0와 동일하게 기본 설정을 한 후, tick기반 simulation을 실행

AiM Simulator Frontend Function

□ Frontend: aim_trace.cpp – tick()

```
1. void tick() override {
2.     if ((remaining_req == false) && (m_trace_reached_EOC == false)) {
3.         AiMTrace_request = get_host_request();
4.         remaining_req = true;
5.     }
6.     if (remaining_req == true) {
7.         bool request_sent = m_memory_system->send(AiMTrace_request);
8.         if (request_sent)
9.             remaining_req = false;
10.    }
11.};
```

request_sent = false가 되기 위해서는 아래의 조건을 만족해야 함

```
src > memory_system > impl > aim_DRAM_system.cpp > {} Ramulator > AiMDRAMSystem > send(Request)
14 namespace Ramulator {
15
16     #define ISR_SIZE (1 << 21)
17     #define MAX_CHANNEL_COUNT 32
18
19     ...
20
187 if (request_queue.size() == ISR_SIZE) {
188     s_ISR_queue_full++;
189     return false;
190 }
```

aim_trace.cpp 다른 함수들도 존재하지만 가장 중요한 tick()을 살펴본다.

▪ tick()

1. Memory System에 보내지 못한 Request가 없거나 trace가 EOC에 도달 X 시
 - Trace file에서 다음 Request를 가져옴
 - EOC = Trace의 종료 (= Simulation의 종료)를 나타내는 Request

2. Memory System에 Request를 보냄 (= trace file에서 가져온 Request)

- m_memory_system->send(AiMTrace_request)가 false이라면,
- 아래 ISR_SIZE 보다 request_queue.size()가 커야 함
- send()에 실패할 경우 Request가 stall이 됨

▪ tick() 외 함수들

- init(): trace file 환경 초기화
- get_host_request(): Trace File의 Line들을 Decoding (= Instruction Decode)
- receive(): EOC 완료 callback (EOC command: simulation 완료를 trigger)
- is_finished(): Simulation 종료 판정
(receive()에서 return true시 is_finished()가 true가 되는 흐름으로 진행)

AiM Simulator Frontend -> Memory System Function

❑ MemorySystem: aim_dram_system.cpp – send()

```
1. bool send(Request req) override {
2.
3.     ///// - debug by MSKIM in ISL yonsei
4.     m_logger->debug("----- Cycle: {}, Get Host Request: {} -----", m_clk, req.str());
5.     /////
6.
7.     if (request_queue.size() == ISR_SIZE) {
8.         s_ISR_queue_full++;
9.         ///// - debug by MSKIM in ISL yonsei
10.        m_logger->debug("== Cycle: {}, ISR queue full for request: {} ==", m_clk, req.str());
11.        /////
12.        return false;
13.    }
14.    request_queue.push(req);
15.    // m_logger->info("[CLK {}] {} pushed to the queue!", m_clk, req.str());
16.
17.    switch (req.type) {
18.    case Type::AIM: {
19.        s_num_AiM_requests[req.opcode]++;
20.        break;
21.    }
22.    case Type::Read:
23.    case Type::Write: {
24.        s_num_RW_requests[req.type][req.mem_access_region]++;
25.        break;
26.    }
27.    default: {
28.        throw ConfigurationError("AiMDRAMSystem: unknown request type {}!", (int)req.type);
29.        break;
30.    }
31.    }
32.
33.    return true;
34. }
```

CMD 통계는 아래와 같이 report가 출력 됨

```
total_num_AiM_ISR_COPY_GBBK_requests: 1 # total number of AiM ISR_COPY_GBBK requests
total_num_W_MEM_requests: 4 # total number of W MEM requests
total_num_AiM_ISR_WR_SBK_requests: 8 # total number of AiM ISR_WR_SBK requests
total_num_R_GPR_requests: 0 # total number of R GPR requests
total_num_AiM_ISR_WR_BIAS_requests: 1 # total number of AiM ISR_WR_BIAS requests
```

aim_trace.cpp – tick()에서 m_memory_system->send() 호출하므로 살펴본다.

▪ send()

1. request_queue에 자리가 있는 지 확인 후, request를 queue에 저장

- $ISR_SIZE = 1 \ll 21 = 2,097,152$
- queue 저장 거절 시 → 다음 cycle에 재시도
- request_queue에 저장된 req는 m_memory_system – tick()에서 처리

2. Request Type별 통계 계산

- Type::AIM 인 CMD 통계 update

CMD 예시	Statistic Update되는 배열
ISR_WR_SBK	s_num_AiM_requests[Opcode::ISR_WR_SBK]
ISR_MAC_ABK	s_num_AiM_requests[Opcode::ISR_MAC_ABK]
ISR_EOC	s_num_AiM_requests[Opcode::ISR_EOC]

- Type::Read 및 Type::Write인 CMD 통계 update

```
// 예: GPR에서 읽은 요청 개수
s_num_RW_requests[Type::Read][MemAccessRegion::GPR]++;

// 예: CFR에 쓴 요청 개수
s_num_RW_requests[Type::Write][MemAccessRegion::CFR]++;

// 예: MEM에서 읽은 요청 개수
s_num_RW_requests[Type::Read][MemAccessRegion::MEM]++;
```

AiM Simulator PIM CMDs Structure

□ base/request.h

```
1. // Basic request id convention
2. // 0 = Read, 1 = Write. The device spec defines all others
3. enum class Type {
4.     Read = 0,
5.     Write = 1,
6.     RefAllBank = 2,
7.     RefSingleBank = 3,
8.     AIM = 4,
9.     MAX = 5
10. };
11.
12. enum class MemAccessRegion {
13.     MIN,
14.     GPR,
15.     CFR,
16.     MEM,
17.     MAX
18. };
19.
20. // ISR Command Opcodes
21. enum class Opcode {
22.     MIN = 0,
23.     ISR_WR_SBK = 1, // Write [op_size * 256 bits] from [GPR * 32] to [a single bank] of channel [#channel_address]
24.     ISR_WR_GB = 2, // Write the same [op_size * 256 bits] starting from [GPR * 32]
25.                     // to the [Global Buffer] of [channel_mask] channels
26.                     // [NOT_IMPLEMENTED] source could also be [host]
27.     ISR_WR_BIAS = 3, // Write [opsize * 16 x 16-bits] bits from [GPR * 32]
28.                     // to [MAC accumulator of 16 banks] of [opsize or channel_mask] channels
29.     ISR_WR_AFLUT = 4, // Write activation function data from [host] to [?]
30.     ISR_RD_MAC = 5, // Read data from [MAC accumulator of all banks] to [GPR]
31.     ISR_RD_AF = 6, // Read data from [AF results of all banks] to [GPR]
32.     ISR_RD_SBK = 7, // [NOT_LISTED] Read data from [a single bank] to [host]
33.     ISR_COPY_BKGB = 8, // Copy data from [a single bank] to [the Global Buffer]
34.     ISR_COPY_GBBK = 9, // Copy data from [the Global Buffer] to [a single bank]
35.     ISR_MAC_SBK = 10, // Perform MAC operation between [a single bank] and [Global Buffer]
36.     ISR_MAC_ABK = 11, // Perform MAC operation between [all banks] and [Global Buffer]
37.     ISR_AF = 12, // Perform Activation Function operation on [all banks]
38.     ISR_EWMUL = 13, // Element wise multiplication between 2 banks of 1 or all bank group(s)
39.     ISR_EWADD = 14, // Element wise multiplication between 2 GPR addresses
40.     ISR_WR_ABK = 15, // Write [16 x 16 bits] from [GPR * 32] to [16 banks] of channel [#channel_address]
41.     ISR_EOC = 16, // End of compute for the current kernel
42.     ISR_SYNC = 17,
43.     MAX = 18
44.     // ISR_WR_HBK, // [NOT_IMPLEMENTED] Write data from [GPR] to [8 banks]
45.     // ISR_WR_ABK, // [NOT_IMPLEMENTED] Write data from [GPR] to [all banks]
46.     // ISR_WR_GPR, // [NOT_IMPLEMENTED] Write data from [host] to [GPR]
47.     // ISR_MAC_HBK, // [NOT_IMPLEMENTED] Perform MAC operation between [8 banks] and [Global Buffer]
48. };
```

본격적으로 Memory System을 살피기 전에, AiM의 CMD가 어떻게 구성되어 있는지 확인

1. Request의 Type을 알려줌. (가장 큰 범위라고 생각하면 됨)

- Read: Normal DRAM Read Operation Request
- Write: Normal DRAM Write Operation Request
- AIM: DRAM Processing in Memory Request
- RefAllBank, RefSingleBank: Not Used (선언만 되어 있음. 동작 구현 없음)

2. 해당 Request가 어느 Memory Region에 Access하는지를 알려줌

- GPR: General Purpose Registers
- CFR: Configuration Registers
- MEM: Memory(DRAM Bank)
- ex. AiM WR_SBK 0 1 0 0 : Write 256bit from GPR[0] to channel[0] bank[0] row[0]
즉, Type = Write, MemAccessRegion = MEM이 됨 (Memory에 Write를 하므로)

3. PIM Commands의 Opcodes를 enum 형식으로 정리

AiM Simulator PIM CMDs Structure

□ base/request.h

```
1. struct Request {
2.     Addr_t addr = -1;
3.     Data_t data;
4.     AddrVec_t addr_vec{};
5.     int host_req_id = -1;
6.     int AiM_req_id = -1;
7.
8.     Type type = Type::MAX;
9.
10.    MemAccessRegion mem_access_region = MemAccessRegion::MAX;
11.
12.    Opcode opcode = Opcode::MAX;
13.
14.    // [NOT_IMPLEMENTED] Increment order for ISR_WR_SBK, ISR_WR_HBK, and ISR_WR_ABK operations
15.    // struct IncrementOrder {
16.    //     enum : int {
17.    //         IO_ROW_COL, // {ROW_ADDR[13:0], COL_ADDR[5:0]}++
18.    //         IO_BK_COL,  // {BK_ADDR[3:0], COL_ADDR[5:0]}++
19.    //         IO_COL_CH,  // {COL_ADDR[3:0], BK_ADDR[5:0]}++
20.    //         MAX
21.    //     };
22.    // };
23.
24.    int32_t opsize = -1;
25.
26.    // [NOT_IMPLEMENTED] Source of ISR_WR_GB is host (false) or GPR (true)
27.    // bool use_GPR = false;
28.
29.    // GPR address 0 USED for: ISR_WR_SBK, ISR_WR_ABK, ISR_WR_GB, ISR_WR_BIAS, ISR_RD_MAC, ISR_EWADD
30.    // GPR address 1 USED for: ISR_EWADD
31.    Addr_t GPR_addr_0 = -1;
32.    Addr_t GPR_addr_1 = -1;
33.
34.    // This request will be broadcasted/multicast to the channels
35.    // whose bit is set in channel mask.
36.    // NOT USED in ISR_EWADD.
37.    // Channel mask must show 1 channel in ISR_WR_ABK ISR
38.    int64_t channel_mask = -1;
39.}
```

```
40. // This request will be sent to a specific bank. USED only in single-bank ISRs, i.e.,
41. // ISR_WR_SBK, ISR_RD_SBK, ISR_COPY_BKGB, ISR_COPY_GBBK, ISR_MAC_SBK, and ISR_EWMUL
42. int16_t bank_index = -1;
43.
44. // Bank row and column address, NOT USED for the operations without DRAM bank source/dst:
45. // ISR_WR_BIAS, ISR_WR_AFLUT, ISR_RD_MAC, ISR_RD_AF, ISR_AF, and ISR_EWADD
46. // In addition, row_addr is NOT USED for ISR_WR_GB
47. int32_t row_addr = -1;
48. int32_t col_addr = -1;
49.
50. // Thread (register) index (0 or 1) for MAC and AF results
51. int8_t thread_index = -1;
52.
53. // Broadcast only USED for ISR_MAC_SBK and ISR_MAC_ABK
54. // vector data for MAC is from GB (0) or next bank (1)
55. int16_t broadcast = -1;
56.
57. // AFM only USED for ISR_AF
58. // Activation Function mode selects AF (0-7)
59. int16_t afm = -1;
60.
61. // ewmul bank group only USED for ISR_MAC_ABK
62. // EWMUL in one bank group (0) or all bank groups (1)
63. int16_t ewmul_bg = -1;
64.
65. int source_id = -1; // An identifier for where the request is coming from (e.g., which core)
66.
67. int command = -1; // The command that need to be issued to progress the request
68. int final_command = -1; // The final command that is needed to finish the request
69.
70. Clk_t arrive = -1; // Clock cycle when the request arrive at the memory controller
71. Clk_t issue = -1; // Clock cycle when the request issued at the memory controller
72. Clk_t depart = -1; // Clock cycle when the request depart the memory controller
73.
74. int type_id = -1;
75.
76. std::function<void(Request &)> callback;
77.
78. Request(Addr_t addr, int type_id);
79. Request(AddrVec_t addr_vec, int type_id);
80. Request(Addr_t addr, int type_id, int source_id, std::function<void(Request &)> callback);
81. // Request(Request const &req);
82.
83. std::string str();
84.
85. bool is_reader();
86. };
```

• 다음 page에 절반씩 나눠서 설명

AiM Simulator PIM CMDs Structure

□ base/request.h

```
1. struct Request {
2.     Addr_t addr = -1;
3.     Data_t data;
4.     AddrVec_t addr_vec{};
5.     int host_req_id = -1;
6.     int AiM_req_id = -1;
7.
8.     Type type = Type::MAX;
9.
10.    MemAccessRegion mem_access_region = MemAccessRegion::MAX;
11.
12.    Opcode opcode = Opcode::MAX;
13.
14.    // [NOT_IMPLEMENTED] Increment order for ISR_WR_SBK, ISR_WR_HBK, and ISR_WR_ABK operations
15.    // struct IncrementOrder {
16.    //     enum : int {
17.    //         IO_ROW_COL, // {ROW_ADDR[13:0], COL_ADDR[5:0]}++
18.    //         IO_BK_COL,  // {BK_ADDR[3:0], COL_ADDR[5:0]}++
19.    //         IO_COL_CH,  // {COL_ADDR[3:0], BK_ADDR[5:0]}++
20.    //         MAX
21.    //     };
22.    // };
23.
24.    int32_t opsize = -1;
25.
26.    // [NOT_IMPLEMENTED] Source of ISR_WR_GB is host (false) or GPR (true)
27.    // bool use_GPR = false;
28.
29.    // GPR address 0 USED for: ISR_WR_SBK, ISR_WR_ABK, ISR_WR_GB, ISR_WR_BIAS, ISR_RD_MAC, ISR_EWADD
30.    // GPR address 1 USED for: ISR_EWADD
31.    Addr_t GPR_addr_0 = -1;
32.    Addr_t GPR_addr_1 = -1;
33.
34.    // This request will be broadcasted/multicast to the channels
35.    // whose bit is set in channel mask.
36.    // NOT USED in ISR_EWADD.
37.    // Channel mask must show 1 channel in ISR_WR_ABK ISR
38.    int64_t channel_mask = -1;
39.}
```

- struct Request: 실제 Memory Request 작업 Instance

addr	AiM trace의 경우 이 addr은 CFR write/read에만 사용 (CFR CMD의 host_req.addr는 0/1/2)
data	CFR의 configuration data (ex. CFR_id = 0 일 때, data = 1 이면 MAC from global buffer)
addr_vec{}	addr_vec는 [channel] [rank] [bankgroup] [bank] [row] 을 저장하기 위한 것 (aim_DRAM_system.cpp - apply_addr_map()에서 Request를 Decoding하여 채움)
host_req_id	Host가 Request를 보낼 때마다 +1씩 increment
AiM_req_id	Type = AiM Request인 Request를 보낼 때마다 +1씩 increment
type	지원되는 Request Type = Read / Write / AiM
mem_access_region	지원되는 Memory Access Region = GPR / CFR / MEM
opcode	ISR_WR_SBK, ISR_WR_GB, ... , ISR_EOC 등의 Instruction의 opcode
opsiz	몇 개의 256bit data를 처리할 것인지, 아래는 opsize = 2인 Request의 예시 (ex. AiM WR_GB 2 2 15 = Write [2-data = 512 bits] from GPR[2-3] to channels [0-3] global buffers)
GPR_addr_0	GPR address를 가지고 있는 변수 (ex. AiM WR_GB 1 2 15의 경우 GPR[2]를 address로 가짐)
GPR_addr_1	또 다른 GPR address가 필요할 시, 해당 Address를 가지고 있는 변수 (ex. AiM EWADD)
channel_mask	총 32bit로 나타낼 수 있으며, 32개의 channel의 access 여부를 관리함 (ex. AiM WR_SBK 0 1 0 0 = AiM [opcode] [GPR_id_0] [channel_mask] [bank_id] [row_id])

AiM Simulator PIM CMDs Structure

❑ base/request.h

```

40. // This request will be sent to a specific bank. USED only in single-bank ISRs, i.e.,
41. // ISR_WR_SBK, ISR_RD_SBK, ISR_COPY_BKGB, ISR_COPY_GBBK, ISR_MAC_SBK, and ISR_EWMUL
42. int16_t bank_index = -1;
43.
44. // Bank row and column address, NOT USED for the operations without DRAM bank source/dst:
45. // ISR_WR_BIAS, ISR_WR_AFLUT, ISR_RD_MAC, ISR_RD_AF, ISR_AF, and ISR_EWADD
46. // In addition, row_addr is NOT USED for ISR_WR_GB
47. int32_t row_addr = -1;
48. int32_t col_addr = -1;
49.
50. // Thread (register) index (0 or 1) for MAC and AF results
51. int8_t thread_index = -1;
52.
53. // Broadcast only USED for ISR_MAC_SBK and ISR_MAC_ABK
54. // vector data for MAC is from GB (0) or next bank (1)
55. int16_t broadcast = -1;
56.
57. // AFM only USED for ISR_AF
58. // Activation Function mode selects AF (0-7)
59. int16_t afm = -1;
60.
61. // ewmul bank group only USED for ISR_MAC_ABK
62. // EWMUL in one bank group (0) or all bank groups (1)
63. int16_t ewmul_bg = -1;
64.
65. int source_id = -1; // An identifier for where the request is coming from (e.g., which core)
66.
67. int command = -1; // The command that need to be issued to progress the request
68. int final_command = -1; // The final command that is needed to finish the request
69.
70. Clk_t arrive = -1; // Clock cycle when the request arrive at the memory controller
71. Clk_t issue = -1; // Clock cycle when the request issued at the memory controller
72. Clk_t depart = -1; // Clock cycle when the request depart the memory controller
73.
74. int type_id = -1;
75.
76. std::function<void(Request &)> callback;
77.
78. Request(Addr_t addr, int type_id);
79. Request(AddrVec_t addr_vec, int type_id);
80. Request(Addr_t addr, int type_id, int source_id, std::function<void(Request &)> callback);
81. // Request(Request const &req);
82.
83. std::string str();
84.
85. bool is_reader();
86. };

```

bank_index	Bank의 address를 가지고 있는 변수
row_addr	Row의 address를 가지고 있는 변수
col_addr	Column의 address를 가지고 있는 변수 (AiM simulation에서는 opsize에 따라 +1 increment 하며 서로 다른 GPR을 접근 or 서로 다른 column access한다는 것을 구분하는 정도의 동작 구현을 위해서만 사용)
thread_index	사용되지 않음
broadcast	broadcast = 0이면 MAC from next bank, 1이면 MAC from next global buffer
afm	0-7의 Activation Function 중 어떤 것을 선택하는지 저장하는 변수 (ex. W CFR 2 1로 설정됨)
ewmul_bg	ewmul_bg = 0이면, EWMUL in all bankgroups, 1이면, EWMUL only 1 bankgroup
source_id	AiM의 frontend는 multi-core CPU를 modeling X -> 사용되지 않음
command	Request를 수행하기 위해 지금 실행되고 있는 command (ex. Read Request = ACT -> RD command 순)
final_command	해당 Request가 완료처리 될 수 있는 마지막 command (ex. Read Request = RD가 final)
arrive	memory controller에 request가 도착한 cycle
issue	memory controller에서 request의 1번째 prerequisite cmd가 실행된 cycle - 1 (ex. Read Request 에서 row가 opened가 아니면, ACT가 prerequisite로 실행됨)
depart	memory controller에 request가 완전히 완료되는 cycle
type_id	Type에 따라 숫자로 저장함 (Read = 0, Write = 1, AIM = 4) [simul 동작에서 사용되지 않음]
is_reader()	Request가 Read를 하는 Request인지, Write를 하는 Request인지 판단 (AiM Request 중 Read Opeartion(= RD_MAC/RD_AF)를 하는 경우, 16개의 BANK의 Accumulator reg를 읽음. RAW hazard를 방지하기 위해 AiM_DMA_blocking = true로 설정되고 해당 Request가 완료될 때까지 다른 Request가 실행되지 못함)

AiM Simulator PIM CMDs Structure

□ base/request.h

```
1. class AiMISR {
2.
3. public:
4.     enum class Field {
5.         opcode,
6.         GPR_addr_0,
7.         GPR_addr_1,
8.         channel_mask,
9.         bank_index,
10.        row_addr,
11.        col_addr,
12.        thread_index
13.    };
14.
15.    std::map<Field, std::string> field_to_str = {
16.        {Field::opcode, "opcode"},
17.        {Field::GPR_addr_0, "GPR_addr_0"},
18.        {Field::GPR_addr_1, "GPR_addr_1"},
19.        {Field::channel_mask, "channel_mask"},
20.        {Field::bank_index, "bank_index"},
21.        {Field::row_addr, "row_addr"},
22.        {Field::col_addr, "col_addr"},
23.        {Field::thread_index, "thread_index"},
24.    };
25.
26.    AiMISR(Opcode opcode_ = Opcode::MAX,
27.           std::vector<Field> legal_fields_ = {},
28.           bool channel_count_eq_one_ = false,
29.           bool AiM_DMA_blocking_ = false,
30.           bool require_reg_RW_mod_ = false,
31.           std::string target_level_ = "")
32.        : opcode(opcode_),
33.          legal_fields(legal_fields_),
34.          channel_count_eq_one(channel_count_eq_one_),
35.          AiM_DMA_blocking(AiM_DMA_blocking_),
36.          require_reg_RW_mod(require_reg_RW_mod_),
37.          target_level(target_level_) {}
38.
39.    Opcode opcode;
40.
41.    std::vector<Field> legal_fields;
42.
43.    bool channel_count_eq_one;
44.    bool AiM_DMA_blocking;
45.    bool require_reg_RW_mod;
46.
47.    std::string target_level;
48.
49.    bool is_field_legal(Field field) {
50.        if (std::count(legal_fields.begin(), legal_fields.end(), field))
51.            return true;
52.        return false;
53.    }
54.
55.    template <typename T>
56.    void is_field_value_legal(Field field, T value);
57. };
58.
59.
```

class AiMISR: ISR Command 정의 - 각 ISR opcode가 어떤 구조/특성을 가져야 하는지 정의

AiM Simulator PIM CMDs Structure

□ base/request.h

```
1. class AiMISRInfo {
2. private:
3.     static std::map<std::string, AiMISR> opcode_str_to_aim_ISR;
4.     static std::map<Opcode, std::string> aim_opcode_to_str;
5.
6.     static std::map<std::string, Type> str_to_type;
7.     static std::map<Type, std::string> type_to_str;
8.
9.     static std::map<std::string, MemAccessRegion> str_to_mem_access_region;
10.    static std::map<MemAccessRegion, std::string> mem_access_region_to_str;
11.
12. public:
13.     static void init() {
14.         aim_opcode_to_str[Opcode::ISR_WR_SBK] = "ISR_WR_SBK";
15.         opcode_str_to_aim_ISR["ISR_WR_SBK"] = AiMISR(Opcode::ISR_WR_SBK,
16.             {AiMISR::Field::GPR_addr_0,
17.              AiMISR::Field::channel_mask,
18.              AiMISR::Field::bank_index,
19.              AiMISR::Field::row_addr},
20.             false, // channel_count_eq_one
21.             false, // AiM_DMA_blocking
22.             false, // require_reg_RW_mod
23.             "column" // target_level
24.         );
25.
26.         aim_opcode_to_str[Opcode::ISR_WR_ABK] = "ISR_WR_ABK";
27.         opcode_str_to_aim_ISR["ISR_WR_ABK"] = AiMISR(Opcode::ISR_WR_ABK,
28.             {AiMISR::Field::GPR_addr_0,
29.              AiMISR::Field::channel_mask,
30.              AiMISR::Field::row_addr},
31.             true, // channel_count_eq_one
32.             false, // AiM_DMA_blocking
33.             false, // require_reg_RW_mod
34.             "column" // target_level
35.         );
36.
37.         aim_opcode_to_str[Opcode::ISR_WR_GB] = "ISR_WR_GB";
38.         opcode_str_to_aim_ISR["ISR_WR_GB"] = AiMISR(Opcode::ISR_WR_GB,
39.             {AiMISR::Field::opsize,
40.              AiMISR::Field::GPR_addr_0,
```

class AiMISRInfo: 모든 ISR 정의를 모아둔 Database - 모든 가능한 ISR 정의를 등록하고 관리

• channel_count_eq_one

- true: 이 ISR은 단일 Channel만 허용
- false: Multi-Channel Broad Cast 허용

(ISR_WR_SBK는 channel_count_eq_one = true여야 할 것 같은데, 오타로 보임 - 근거 : trace file 주석)

• AiM_DMA_blocking

- AiM Request 중 Read Opeartion(= RD_MAC/RD_AF)를 하는 경우,
- 16개의 BANK의 Accumulator reg를 읽음.
- RAW hazard를 방지하기 위해 AiM_DMA_blocking = true로 설정
- 이는 해당 Request가 완료될 때까지 다른 Request가 실행되지 못하도록 함

• require_reg_RW_mod = true

- Register 경로(GB, MAC/AF 등) 접근이 필요한 ISR
- Controller는 이를 보고 TMOD로 mode 전환 후 command을 Issue함

AiM Simulator MemorySystem Function

□ MemorySystem: aim_dram_system.cpp – tick()

```
1. void tick() override {
2.
3.     bool was_AiM_request_remaining = false;
4.     bool is_AiM_request_remaining = false;
5.     for (int channel_id = 0; channel_id < MAX_CHANNEL_COUNT; channel_id++) {
6.         while (remaining_AiM_requests[channel_id].empty() == false) {
7.             was_AiM_request_remaining = true;
8.             m_logger->info("[CLK {}] 0- Sending {} to channel {}", m_clk, remaining_AiM_requests[channel_id].front().str(), channel_id);
9.             if (m_controllers[channel_id]->send(remaining_AiM_requests[channel_id].front()) == false) {
10.                 m_logger->info("[CLK {}] 0- failed", m_clk, channel_id);
11.                 is_AiM_request_remaining = true;
12.                 break;
13.             }
14.             remaining_AiM_requests[channel_id].pop();
15.         }
16.     }
17.
18.     if (stalled_AiM_requests == 0) {
19.         if (was_AiM_request_remaining == true) {
20.             if (is_AiM_request_remaining == false) {
21.                 Request host_req = request_queue.front();
22.                 if (host_req.callback)
23.                     host_req.callback(host_req);
24.                 request_queue.pop();
25.             }
26.             } else if (request_queue.empty() == false) {
27.                 Request host_req = request_queue.front();
28.                 // m_logger->info("[CLK {}] Decoding {}...", m_clk, host_req.str());
29.                 bool all_AiM_requests_sent = true;
30.
31.                 switch (host_req.type) {
32.                     case Type::AiM: {
33.                         Opcode opcode = host_req.opcode;
34.                         auto opcode = host_req.opsize;
35.                         int64_t ch_mask = host_req.channel_mask;
36.                         uint8_t channel_count = CountSetBit(ch_mask); // number of channels involved in this request
37.                         Request aim_req = host_req;
38.                         if (aim_req.opcode == Opcode::ISR_RD_SBK) {
39.                             aim_req.type = Type::Read;
40.                             aim_req.mem_access_region = MemAccessRegion::MEM;
41.                             aim_req.opcode = Opcode::MIN;
42.                         } else if (aim_req.opcode == Opcode::ISR_WR_SBK) {
43.                             aim_req.type = Type::Write;
44.                             aim_req.mem_access_region = MemAccessRegion::MEM;
45.                             aim_req.opcode = Opcode::MIN;
46.                         }
47.                     }
48.                 }
49.             }
50.         }
51.     }
52. }
```

1. 실패 했었던 Request가 존재 시 해당 Request send 먼저 시도

- 실패 했었던 Request를 다시 send 했음에도, 다시 한번 실패 시,
- is_AiM_request_remaining = true임
- 따라서, controller에게 send하지 않으며,
- request_queue에서 새로운 Request도 불러오지 않음
- 즉, 1개가 실패하면 그 1개를 성공할 때까지 시도함
- *In-Order Processing*으로 구성되었다고 볼 수 있음
- 따라서, Processing 중 *Timing Constraint*에 의한 latency가 큼

2. 실패 했었던 Request를 보낸 cycle일 때,

- request_queue에 아직 실패 했었던 Request가 남아있음
- (앞의 code를 통해서 send한 것은 channel별로 쪼개진 request일 뿐)
- (사실상 같은 request가 request_queue에 남아 있음)
- 앞의 code를 통해 성공적으로 send를 했기에,
- host를 callback해야 하는 동작이 존재하면, callback을 하고,
- request_queue에서 완전히 삭제

보통 RD/WR request 가 남아있으면 AiM request를 Controller에서 거부(반대도 성립)

```
47 [AIMDRAMSystem] [debug] ----- Cycle: 20, Get Host Request: Request[Type(ISR_WR_6B), OpSize(2), GPR0(2), CHMask(15), hostID(20), ] -----
48 [AIMDRAMSystem] [info] [CLK 20] 1- Sending Request[Type(ISR_WR_6B), OpSize(2), GPR0(2), CHMask(15), CO(0), hostID(20), AiMID(12), Address Vec(31, N/A, N/A, N/A, 0), ]
49 [AIMDRAMController[31]] [debug] Cannot send AiM request while write or read buffer is not empty
50 [AIMDRAMController[31]] [debug] -- Write Buffer[0]: Request[Type(Write), Region(MEM), hostID(16), AiMID(8), Address Vec(31, 0, 0, 0, 0), ]
```


AiM Simulator MemorySystem Function

MemorySystem: aim_dram_system.cpp – tick()

```
1. void tick() override {
2.
26. } else if (request_queue.empty() == false) {
27.     Request host_req = request_queue.front();
28.     // m_logger->info("[CLK {}] Decoding {}", m_clk, host_req.str());
29.     bool all_Aim_requests_sent = true;
30.
31.     switch (host_req.type) {
32.     case Type::ATM: {
33.         Opcode opcode = host_req.opcode;
34.         auto opsize = host_req.opsize;
35.         int64_t ch_mask = host_req.channel_mask;
36.         uint8_t channel_count = CountSetBit(ch_mask); // number of channels involved in this request
37.         Request aim_req = host_req;
38.         if (aim_req.opcode == Opcode::ISR_RD_SBK) {
39.             aim_req.type = Type::Read;
40.             aim_req.mem_access_region = MemAccessRegion::MFM;
41.             aim_req.opcode = Opcode::MTN;
42.         } else if (aim_req.opcode == Opcode::ISR_WR_SBK) {
43.             aim_req.type = Type::Write;
44.             aim_req.mem_access_region = MemAccessRegion::MFM;
45.             aim_req.opcode = Opcode::MTN;
46.         }
47.
48.         switch (opcode) {
49.         case Opcode::ISR_WR_SBK:
50.         case Opcode::ISR_WR_GB:
51.         case Opcode::ISR_WR_BIAS:
52.         case Opcode::ISR_RD_MAC:
53.         case Opcode::ISR_RD_AF:
54.         case Opcode::ISR_RD_SBK:
55.         case Opcode::ISR_COPY_BKGB:
56.         case Opcode::ISR_COPY_GBBK:
57.         case Opcode::ISR_MAC_SBK:
58.         case Opcode::ISR_MAC_ABK:
59.         case Opcode::ISR_AF:
60.         case Opcode::ISR_EWMUL:
61.         case Opcode::ISR_WR_ABK: {
62.             // Decoding opcode
63.             AiMISR aim_isr = AiMISRInfo::convert_Aim_opcode_to_Aim_ISR(opcode);
64.
65.             if (aim_isr.Aim_DMA_blocking) {
66.                 aim_req.callback = callback;
67.             }
68.
69.             if (aim_isr.channel_count_eq_one) {
70.                 if (channel_count != 1) {
71.                     // comment by MSKIM in ISL yonsei
72.                     // 미완성된 Error Message 로 보임 - request.h 의 ISR_WR_SBK 는 channel_count_eq_one = false 로 되어 있음
73.                     // -> 근데 ISR_WR_SBK 가 channel_count_eq_one = true 여야 의도하는 바일 듯 함
74.                     // -> throw ConfigurationError("AIMDRAMSystem: channel mask ({}) of ISR_WR_SBK must specify only 1 channel!", ch_mask);
75.                     // ->
76.                 }
77.             }
78.
79.             if (opcode == Opcode::ISR_AF) {
80.                 aim_req.afm = CFR_values[CFR::AFM];
81.                 aim_req.row_addr = (1 << 29) + aim_req.afm;
82.             }
83.
84.             if ((host_req.opcode == Opcode::ISR_MAC_ABK) ||
85.                 (host_req.opcode == Opcode::ISR_MAC_SBK)) {
86.                 aim_req.broadcast = CFR_values[CFR::BROADCAST];
87.             }
88.
89.             if (host_req.opcode == Opcode::ISR_MAC_ABK) {
90.
```

3. 실패 했었던 Request를 보낸 cycle 아닐 때,

- request_queue에서 새로운 request를 가져옴 (FIFO 방식)
- 해당 request를 Decoding함

3-1. WR_SBK 및 RD_SBK 처리

- WR_SBK 및 RD_SBK의 경우 Normal DRAM Write/Read로 변경
- Timing Constraints나 Scheduling 처리가 일반 Read/Write와 동일해짐

3-2. Type:AIM인 Request

- 그 후 Type::AIM인 request들을 아래 **AiMISR형식**으로 바꿈
- 어떤 Request인지에 따라, struct Request의 opcode, opsize, row_addr 등의 정보를 채움
- 즉, request를 decoding하며, 필요한 정보들을(= struct Request에 선언된 변수들) 적절히 그대로 or 연산하여 채워 넣음

```
static void init() {
    aim_opcode_to_str[Opcode::ISR_WR_SBK] = "ISR_WR_SBK";
    opcode_str_to_aim_isr["ISR_WR_SBK"] = AiMISR(opcode::ISR_WR_SBK,
        {AiMISR::Field::GPR_addr_0,
         AiMISR::Field::channel_mask,
         AiMISR::Field::bank_index,
         AiMISR::Field::row_addr},
        false, // channel_count_eq_one
        false, // Aim_DMA_blocking
        false, // require_reg_rw_mod
        "column" // target_level
    );
}
```

의도된 것인지 모르겠지만, RD_SBK와 WR_SBK만 Type을 바꿈
RD_ABK와 WR_ABK는 일부러 Type을 바꾸지 않은 것인가?
-> Yes, ABK의 경우 All Bank를 접근하기에 Normal DRAM operation과 다름

❑ MemorySystem: aim_dram_system.cpp – tick()

...

4. dram과 controller가 매 cycle해야 하는 동작을 수행

- ```
src > dram_controller > impl > AIM_dram_controller.cpp > {} Ramulator > AiMDRAMController > priority_send(Request &)
```
- ```
229 void tick() override {
230     m_clk++;
231     // if ((m_clk == 1) || (m_clk % 1000 == 0))
232     //     m_logger->info("[CLK {}]", m_clk);
233
234     // 1. Serve completed reads
235     serve_completed_reqs();
236
237     m_refresh->tick();
238
239     // 2. Try to find a request to serve.
240     ReqBuffer::iterator req_it;
241     ReqBuffer *buffer = nullptr;
242     bool request_found = schedule_request(req_it, buffer);
243
244     // // 3. Update all plugins
245     // for (auto plugin : m_plugins) {
246     //     plugin->update(request_found, req_it);
247     // }
248
249     // 4. Finally, issue the commands to serve the request
250     if (request_found) {
251         if ((req_it->opcode == Opcode::ISR_EOC) || (req_it->opcode == Opcode::ISR_SYNC)) {
```

```
src > dram > impl > GDDR6.cpp > {} Ramulator > GDDR6 > m_channels
5 namespace Ramulator {
7 class GDDR6 : public IDRAM, public Implementation {
452
453 public:
454     void tick() override {
455         m_clk++;
456     };
}
```

- ```
430 inline static constexpr ImplDef m_states = {
431 "Opened", "Closed", "PowerUp", "N/A"};
```

24

# AiM Simulator MemoryController Function

## ❑ MemoryController: AiM\_dram\_controller.cpp – send()

```
1. bool send(Request &req) override {
2. if (req.type == Type::AiM) {
3. if ((m.write_buffer.size() != 0) || (m.read_buffer.size() != 0)) {
4. // debug by MSKIM in ISL yonsei
5. m_logger->debug("Cannot send AiM request while write or read buffer is not empty");
6. int i = 0;
7. for (auto it = m.write_buffer.begin(); it != m.write_buffer.end(); ++it, ++i) {
8. m_logger->debug("-- Write Buffer[{}]: {}", i, it->str());
9. }
10. i = 0;
11. for (auto it = m.read_buffer.begin(); it != m.read_buffer.end(); ++it, ++i) {
12. m_logger->debug("-- Read Buffer[{}]: {}", i, it->str());
13. }
14. // debug by MSKIM in ISL yonsei
15. return false;
16. }
17. req.final_command = m.dram->m_aim_request_translations((int)req.opcode);
18. } else {
19. if (m.aim_buffer.size() != 0) {
20. // debug by MSKIM in ISL yonsei
21. m_logger->debug("Cannot send non-AiM request while AiM buffer is not empty");
22. int i = 0;
23. for (auto it = m.aim_buffer.begin(); it != m.aim_buffer.end(); ++it, ++i) {
24. m_logger->debug("-- AiM Buffer[{}]: {}", i, it->str());
25. }
26. // debug by MSKIM in ISL yonsei
27. return false;
28. }
29. req.final_command = m.dram->m_request_translations((int)req.type);
30. }
31.
32. // Forward existing write requests to incoming read requests
33. if (req.type == Type::Read) {
34. // auto compare_addr = [req](const Request &wreq) {
35. // return wreq.addr == req.addr;
36. // };
37.
38. // debug by MSKIM in ISL yonsei
39. auto compare_addr = [this, &req](const Request &wreq) {
40. // Prefer flat address if both are valid
41. if (wreq.addr != -1 && req.addr != -1) {
42. return wreq.addr == req.addr;
43. }
44.
45. // Fallback to addr_vec comparison (channel-row only)
46. if (wreq.addr_vec.size() == 0 || req.addr_vec.size() == 0) {
47. return false;
48. }
49. };
```

```
50. int row_idx = m.dram->m_levels("row");
51. for (int level_idx = m.dram->m_levels("channel"); level_idx <= row_idx; level_idx++) {
52. if (wreq.addr_vec[level_idx] == -1 || req.addr_vec[level_idx] == -1) {
53. return false;
54. }
55. if (wreq.addr_vec[level_idx] != req.addr_vec[level_idx]) {
56. return false;
57. }
58. }
59. // debug by MSKIM in ISL yonsei
60. // m_logger->debug("Read request {} forwarded", req.str());
61. // debug by MSKIM in ISL yonsei
62. return true;
63. }
64. // debug by MSKIM in ISL yonsei
65. if (std::find_if(m.write_buffer.begin(), m.write_buffer.end(), compare_addr) != m.write_buffer.end()) {
66. // The request will depart at the next cycle
67. req.depart = m.clk + 1;
68. pending_reads.push_back(req);
69. return true;
70. }
71.
72. // Else, enqueue them to corresponding buffer based on request type id
73. bool is_success = false;
74. req.arrive = m.clk;
75. if (req.type == Type::Read) {
76. is_success = m.read_buffer.enqueue(req);
77. } else if (req.type == Type::Write) {
78. is_success = m.write_buffer.enqueue(req);
79. } else if (req.type == Type::AiM) {
80. is_success = m.aim_buffer.enqueue(req);
81. } else {
82. throw std::runtime_error("Invalid request type!");
83. }
84.
85. if (!is_success) {
86. // We could not enqueue the request
87. req.arrive = -1;
88. // debug by MSKIM in ISL yonsei
89. m_logger->debug("Failed to enqueue request of type {}, Buffer is full", (int)req.type);
90. // debug by MSKIM in ISL yonsei
91. return false;
92. }
93.
94. return true;
95.};
```

### 1. Read/Write Buffer ↔ AiM Buffer 상호 독립성 확보

- Read/Write Request가 Read/Write Buffer에 존재 시, Controller는 AiM Request를 거부함 – memory system에서
- AiM Request가 AiM Buffer에 존재 시, 마찬가지로 Read/Write Request를 거부함
- 만약 받을 수 있는 Request라면, Request의 final\_command를 struct Request의 final\_command 변수에 저장해놓음

# AiM Simulator MemoryController Function

## ❑ MemoryController: AiM\_dram\_controller.cpp – send()

```
1. bool send(Request &req) override {
2. if (req.type == Type::AIM) {
3. if ((m_write_buffer.size() != 0) || (m_read_buffer.size() != 0)) {
4. // debug by MSKIM in ISL yonsei
5. m_logger->debug("Cannot send AIM request while write or read buffer is not empty");
6. int i = 0;
7. for (auto it = m_write_buffer.begin(); it != m_write_buffer.end(); ++it, ++i) {
8. m_logger->debug("Write Buffer[{}]: {}", i, it->str());
9. }
10. i = 0;
11. for (auto it = m_read_buffer.begin(); it != m_read_buffer.end(); ++it, ++i) {
12. m_logger->debug("Read Buffer[{}]: {}", i, it->str());
13. }
14. // debug by MSKIM in ISL yonsei
15. return false;
16. }
17. req.final_command = m_dram->m_aim_request_translations((int)req.opcode);
18. } else {
19. if (m_aim_buffer.size() != 0) {
20. // debug by MSKIM in ISL yonsei
21. m_logger->debug("Cannot send non-AIM request while AIM buffer is not empty");
22. int i = 0;
23. for (auto it = m_aim_buffer.begin(); it != m_aim_buffer.end(); ++it, ++i) {
24. m_logger->debug("AIM Buffer[{}]: {}", i, it->str());
25. }
26. // debug by MSKIM in ISL yonsei
27. return false;
28. }
29. req.final_command = m_dram->m_request_translations((int)req.type);
30. }
31.
32. // Forward existing write requests to incoming read requests
33. if (req.type == Type::Read) {
34. // auto compare_addr = [req](const Request &wreq) {
35. // return wreq.addr == req.addr;
36. // };
37.
38. // debug by MSKIM in ISL yonsei
39. auto compare_addr = [this, &req](const Request &wreq) {
40. // Prefer flat address if both are valid
41. if (wreq.addr != -1 && req.addr != -1) {
42. return wreq.addr == req.addr;
43. }
44.
45. // Fallback to addr_vec comparison (channel-row only)
46. if (wreq.addr_vec.size() == 0 || req.addr_vec.size() == 0) {
47. return false;
48. }
49. };
```

```
50. int row_idx = m_dram->m_levels("row");
51. for (int level_idx = m_dram->m_levels("channel"); level_idx <= row_idx; level_idx++) {
52. if (wreq.addr_vec[level_idx] == -1 || req.addr_vec[level_idx] == -1) {
53. return false;
54. }
55. if (wreq.addr_vec[level_idx] != req.addr_vec[level_idx]) {
56. return false;
57. }
58. }
59. // debug by MSKIM in ISL yonsei
60. // m_logger->debug("Read request {} forwarded", req.str());
61. // debug by MSKIM in ISL yonsei
62. return true;
63. }
64.
65. // If (std::find_if(m_write_buffer.begin(), m_write_buffer.end(), compare_addr) != m_write_buffer.end()) {
66. // // The request will depart at the next cycle
67. // req.depart = m_clk + 1;
68. // pending_reads.push_back(req);
69. // return true;
70. // }
71.
72.
73. // Else, enqueue them to corresponding buffer based on request type id
74. bool is_success = false;
75. req.arrive = m_clk;
76. if (req.type == Type::Read) {
77. is_success = m_read_buffer.enqueue(req);
78. } else if (req.type == Type::Write) {
79. is_success = m_write_buffer.enqueue(req);
80. } else if (req.type == Type::AIM) {
81. is_success = m_aim_buffer.enqueue(req);
82. } else {
83. throw std::runtime_error("Invalid request type!");
84. }
85. if (!is_success) {
86. // We could not enqueue the request
87. req.arrive = -1;
88. // debug by MSKIM in ISL yonsei
89. m_logger->debug("Failed to enqueue request of type {}, Buffer is full", (int)req.type);
90. // debug by MSKIM in ISL yonsei
91. return false;
92. }
93.
94. return true;
95. };
```

## 2. RAW hazard 방지를 위해 Data Forwarding 수행하기 위한 compare\_addr function 정의 (이 부분은 기존 구현이 이상하여, 직접 수정했습니다 - 민성)

- DRAM addr\_vec를 channel ~ row level까지 비교 (write buffer에 존재하는 Request들의 addr\_vec과 현재 send된 Request의 addr\_vec 비교)
- 만약 같은 주소를 가지는 write\_request가 존재 시, data forwarding 수행 (= write되어야 하는 data를 나중에 들어온 read request가 가지고 감)
- pending buffer는 완료되기 직전 Request들이 저장되어 있는 buffer (ex. Read Request의 경우 RD command가 issue되면 완료 직전 -> pending으로 옮겨짐 -> read latency만큼 pending에서 기다림 -> latency가 다 해소되면 pop()됨)

# AiM Simulator MemoryController Function

## ❑ MemoryController: AiM\_dram\_controller.cpp – send()

```
1. bool send(Request &req) override {
2. if (req.type == Type::AiM) {
3. if ((m.write_buffer.size() != 0) || (m.read_buffer.size() != 0)) {
4. // debug by MSKIM in ISL yonsei
5. m_logger->debug("Cannot send AiM request while write or read buffer is not empty");
6. int i = 0;
7. for (auto it = m.write_buffer.begin(); it != m.write_buffer.end(); ++it, ++i) {
8. m_logger->debug("-- Write Buffer[{}]: {}", i, it->str());
9. }
10. i = 0;
11. for (auto it = m.read_buffer.begin(); it != m.read_buffer.end(); ++it, ++i) {
12. m_logger->debug("-- Read Buffer[{}]: {}", i, it->str());
13. }
14. // debug by MSKIM in ISL yonsei
15. return false;
16. }
17. req.final_command = m.dram->m.aim_request_translations((int)req.opcode);
18. } else {
19. if (m.aim_buffer.size() != 0) {
20. // debug by MSKIM in ISL yonsei
21. m_logger->debug("Cannot send non-AiM request while AiM buffer is not empty");
22. int i = 0;
23. for (auto it = m.aim_buffer.begin(); it != m.aim_buffer.end(); ++it, ++i) {
24. m_logger->debug("-- AiM Buffer[{}]: {}", i, it->str());
25. }
26. // debug by MSKIM in ISL yonsei
27. return false;
28. }
29. req.final_command = m.dram->m.request_translations((int)req.type);
30. }
31.
32. // Forward existing write requests to incoming read requests
33. if (req.type == Type::Read) {
34. // auto compare_addr = [req](const Request &wreq) {
35. // return wreq.addr == req.addr;
36. // };
37.
38. // debug by MSKIM in ISL yonsei
39. auto compare_addr = [this, &req](const Request &wreq) {
40. // Prefer flat address if both are valid
41. if (wreq.addr != -1 && req.addr != -1) {
42. return wreq.addr == req.addr;
43. }
44.
45. // Fallback to addr_vec comparison (channel-row only)
46. if (wreq.addr_vec.size() == 0 || req.addr_vec.size() == 0) {
47. return false;
48. }
49. };
```

```
50. int row_idx = m.dram->m.levels("row");
51. for (int level_idx = m.dram->m.levels("channel"); level_idx <= row_idx; level_idx++) {
52. if (wreq.addr_vec[level_idx] == -1 || req.addr_vec[level_idx] == -1) {
53. return false;
54. }
55. if (wreq.addr_vec[level_idx] != req.addr_vec[level_idx]) {
56. return false;
57. }
58. }
59. // debug by MSKIM in ISL yonsei
60. // m_logger->debug("Read request {} forwarded", req.str());
61. // debug by MSKIM in ISL yonsei
62. return true;
63. }
64.
65. // If (std::find_if(m.write_buffer.begin(), m.write_buffer.end(), compare_addr) != m.write_buffer.end()) {
66. // // The request will depart at the next cycle
67. // req.depart = m.clk + 1;
68. // pending_reads.push_back(req);
69. // return true;
70. // }
71.
72.
73. // Else, enqueue them to corresponding buffer based on request type id
74. bool is_success = false;
75. req.arrive = m.clk;
76. if (req.type == Type::Read) {
77. is_success = m.read_buffer.enqueue(req);
78. } else if (req.type == Type::Write) {
79. is_success = m.write_buffer.enqueue(req);
80. } else if (req.type == Type::AiM) {
81. is_success = m.aim_buffer.enqueue(req);
82. } else {
83. throw std::runtime_error("Invalid request type!");
84. }
85. if (!is_success) {
86. // We could not enqueue the request
87. req.arrive = -1;
88. // debug by MSKIM in ISL yonsei
89. m_logger->debug("Failed to enqueue request of type {}, Buffer is full", (int)req.type);
90. // debug by MSKIM in ISL yonsei
91. return false;
92. }
93.
94. return true;
95. };
```

### 3. Request가 거부되지 않고, forwarding도 수행되지 않았을 시, 적절한 buffer로 enqueue

- Read/Write/AiM request에 따라 적절한 buffer로 enqueue됨
- 각각의 request buffer의 MAX\_SIZE = 32로, 32개의 Request가 이미 buffer에 쌓여 있으면 return false이고, 다음 cycle에 재시도 됨 (return false 시, memory system에서 실패 했던 Request를 다시 send 하는 logic으로 이어짐)

# AiM Simulator MemoryController Function

## MemoryController: AiM\_dram\_controller.cpp – tick()

```
1. void tick() override {
2. m_clk++;
3. // if ((m_clk == 1) || (m_clk % 1000 == 0))
4. // m_logger->info("[CLK {}]", m_clk);
5.
6. // 1. Serve completed reads
7. serve_completed_reqs();
8.
9. m_refresh->tick();
10.
11. // 2. Try to find a request to serve.
12. ReqBuffer::iterator req_it;
13. ReqBuffer *buffer = nullptr;
14. bool request_found = schedule_request(req_it, buffer);
15.
16. // 3. Update all plugins
17. // for (auto plugin : m_plugins) {
18. // plugin->update(request_found, req_it);
19. // }
20.
21. // 4. Finally, issue the commands to serve the request
22. if (request_found) {
23. if ((req_it->opcode == Opcode::ISR_EOC) || (req_it->opcode == Opcode::ISR_SYNC)) {
24. req_it->depart = m_clk;
25. pending_reads.push_back(*req_it);
26. buffer->remove(req_it);
27. // m_logger->info("[CLK {}] EOC/SYNC ready for callback", m_clk);
28. } else {
29. bool requires_reg_RW_mode = false;
30. if (req_it->type == Type::AIM) {
31. // m_logger->info("Checking AiM request: {}", req_it->str());
32. if (AiMInfo::opcode_requires_reg_RW_mod(req_it->opcode)) {
33. requires_reg_RW_mode = true;
34. }
35. }
36. }
37.
38. if (requires_reg_RW_mode ^ is_reg_RW_mode) {
39. req_it->command = m_dram->m_commands("TMOO");
40. is_reg_RW_mode = !is_reg_RW_mode;
41. }
42.
43. // If we find a real request to serve
44. m_logger->info("[CLK {}] Issuing {} for {}", m_clk, std::string(m_dram->m_commands(req_it->command)).c_str(), req_it->str());
45. if (req_it->issue == -1) {
46. req_it->issue = m_clk - 1;
47. }
48. m_dram->issue_command(req_it->command, req_it->addr_vec);
49. s_num_commands[req_it->command] += 1;
50. }
```

```
51. // If we are issuing the last command, set depart clock cycle and move the request to the pending_reads queue
52. if (req_it->command == req_it->final_command) {
53. int latency = m_dram->m_command_latencies(req_it->command);
54. assert(latency > 0);
55. req_it->depart = m_clk + latency;
56. if (req_it->is_reader()) {
57. pending_reads.push_back(*req_it);
58. } else {
59. pending_writes.push_back(*req_it);
60. }
61. if (req_it->type == Type::AIM) {
62. // - commented by MSKIM in ISL yonsei
63. // AiM_cycles update Logic 이 이상함
64. // 원래 code : m_clk - req_it->issue 의 경우 prerequisite command issue 시간만 포함됨
65. // 만약 scheduling buffer 대기시간 포함 & final_req의 latency 포함 : req_it->depart - req_it->arrive
66. // 만약 scheduling buffer 대기시간 포함 & final_req의 latency 미포함 : m_clk - req_it->arrive
67. // 만약 scheduling buffer 대기시간 미포함 & final_req의 latency 포함 : req_it->depart - req_it->issue
68. // 만약 scheduling buffer 대기시간 미포함 & final_req의 latency 미포함 : m_clk - req_it->issue
69. s_num_AiM_cycles[req_it->opcode] += (m_clk - req_it->issue);
70. } else {
71. // - commented by MSKIM in ISL yonsei
72. // RW_cycles update Logic 도 같은 이유로 이상함
73. s_num_RW_cycles[req_it->type] += (m_clk - req_it->issue);
74. }
75. // else if (req_it->type == Type::Write) {
76. // // TODO: Add code to update statistics
77. // }
78. buffer->remove(req_it);
79. if (req_it->type != Type::AIM) {
80. if (m_dram->m_command_meta(req_it->command).is_opening) {
81. m_active_buffer.enqueue(*req_it);
82. buffer->remove(req_it);
83. }
84. }
85. }
86. else if (m_read_buffer.size() == 0 && m_write_buffer.size() == 0 && m_aim_buffer.size() == 0 && pending_reads.size() == 0 && pending_writes.size() == 0) {
87. // if (m_channel_id == 0)
88. // m_logger->info("[CLK {}] CH0 IDLE", m_clk);
89. s_num_idle_cycles += 1;
90. }
91. // - added by MSKIM in ISL yonsei
92. else {
93. s_num_not_issued_cycles += 1;
94. }
95. // - - - - -
96. if (m_dram->m_open_rows[m_channel_id] == 0) {
97. s_num_precharged_cycles += 1;
98. } else {
99. s_num_active_cycles += 1;
100. }
101.
102. };
```

### 1. Pending buffer에 있는 Request들을 완료 처리 & Refresh 관리 & Request Scheduling

- 1.1. Pending buffer에 있는 Request들이 완료될 수 있는지 확인하고 완료 가능 시, 완료 처리를 함 (frontend callback 필요 시 수행 후, pending.pop() 수행)
- 1.2. DRAM refresh는 일정 주기마다 수행됨 -> 이 주기를 확인하고 refresh할 주기가 되었으면, priority buffer로 refresh request를 enqueue
- 1.3. Request를 Scheduling하여 issue가능한 Request 존재 시, 적절한 Request를 가져옴

# AiM Simulator MemoryController Function

## MemoryController: AiM\_dram\_controller.cpp – tick()

```
1. void tick() override {
2. m_clk++;
3. // if ((m_clk == 1) || (m_clk % 1000 == 0))
4. // m_logger->info("[CLK {}]", m_clk);
5.
6. // 1. Serve completed reads
7. serve_completed_reqs();
8.
9. m_refresh->tick();
10.
11. // 2. Try to find a request to serve.
12. ReqBuffer::iterator req_it;
13. ReqBuffer *buffer = nullptr;
14. bool request_found = schedule_request(req_it, buffer);
15.
16. // 3. Update all plugins
17. for (auto plugin : m_plugins) {
18. plugin->update(request_found, req_it);
19. }
20.
21. // 4. Finally, issue the commands to serve the request
22. if (request_found) {
23. if ((req_it->opcode == Opcode::ISR_EOC) || (req_it->opcode == Opcode::ISR_SYNC)) {
24. req_it->depart = m_clk;
25. pending_reads.push_back(*req_it);
26. buffer->remove(req_it);
27. // m_logger->info("[CLK {}] EOC/SYNC ready for callback", m_clk);
28. } else {
29. bool requires_reg_rw_mode = false;
30. if (req_it->type == Type::AIM) {
31. // m_logger->info("Checking AIM request: {}", req_it->str());
32. if (AiMISInfo::opcode_requires_reg_rw_mod(req_it->opcode)) {
33. requires_reg_rw_mode = true;
34. }
35. }
36.
37. if (requires_reg_rw_mode ^ is_reg_rw_mode) {
38. req_it->command = m_dram->m_commands("TMOO");
39. is_reg_rw_mode = !is_reg_rw_mode;
40. }
41.
42. // If we find a real request to serve
43. m_logger->info("[CLK {}] Issuing {} for {}", m_clk, std::string(m_dram->m_commands(req_it->command)).c_str(), req_it->str());
44. if (req_it->issue == -1) {
45. req_it->issue = m_clk - 1;
46. }
47.
48. m_dram->issue_command(req_it->command, req_it->addr_vec);
49. s_num_commands[req_it->command] += 1;
50. }
51. }
52.}
```

```
1. void serve_completed_reqs() {
2. if (pending_reads.size()) {
3. // Check the first pending reads request
4. auto &req = pending_reads[0];
5. if (req.depart <= m_clk) {
6. // Request received data from dram
7.
8. if (((req.opcode != Opcode::ISR_EOC) && (req.opcode != Opcode::ISR_SYNC)) ||
9. (pending_writes.size() == 0)) {
10.
11. if (req.callback) {
12. // If the request comes from outside (e.g., processor), call its callback
13. // m_logger->info("[CLK {}] Calling back {}!", m_clk, req.str());
14. req.callback(req);
15. }
16. // else {
17. // m_logger->info("[CLK {}] Warning: {} doesn't have callback set but it is in the pending_reads queue!", m_clk, req.str());
18. // }
19. // Finally, remove this request from the pending_reads queue
20. pending_reads.pop_front();
21.
22. }
23.
24. auto write_req_it = pending_writes.begin();
25. while (write_req_it != pending_writes.end()) {
26. if (write_req_it->depart <= m_clk) {
27. // Remove this write request
28. // m_logger->info("[CLK {}] Finished {}!", m_clk, write_req_it->str());
29. write_req_it = pending_writes.erase(write_req_it);
30. } else {
31. ++write_req_it;
32. }
33. }
34. }
35. }
36. }
37.}
```

### 1. Pending buffer에 있는 Request들을 완료 처리 & Refresh 관리 & Request Scheduling – 세부 사항 1 : serve\_complete\_reqs()

- Pending Buffer에 Read Request들이 존재 시, 완료될 수 있는 지 확인 후, 완료 처리 (함수 호출 1번 당 1개의 Read Request만 완료 처리 가능)  
(Data Bus 완료는 cycle당 1건, DRAM의 read data 반환은 내부적으로 Serialized 되어 Bus 점유, 이 model은 1 cycle에 하나의 read 완료만 처리하도록 단순화)
- 그 후, Pending Buffer에 존재하는 Write Request 중 완료될 수 있는 것을 모두 처리 (Write Request는 Data Bus 사용 X이므로, 여러 개 완료 가능)



# AiM Simulator MemoryController Function

## ❑ MemoryController: AiM\_dram\_controller.cpp – tick()

```
1. void tick() override {
2. m_clk++;
3. // if ((m_clk == 1) || (m_clk % 1000 == 0))
4. // m_logger->info("[CLK {}]", m_clk);
5.
6. // 1. Serve completed reads
7. serve_completed_reqs();
8.
9. m_refresh->tick();
10.
11. // 2. Try to find a request to serve.
12. ReqBuffer::iterator req_it;
13. ReqBuffer *buffer = nullptr;
14. bool request_found = schedule_request(req_it, buffer);
15.
16. // 3. Update all plugins
17. for (auto plugin : m_plugins) {
18. plugin->update(request_found, req_it);
19. }
20.
21. // 4. Finally, issue the commands to serve the request
22. if (request_found) {
23. if ((req_it->opcode == Opcode::ISR_EOC) || (req_it->opcode == Opcode::ISR_SYNC)) {
24. req_it->depart = m_clk;
25. pending_reads.push_back(*req_it);
26. buffer->remove(req_it);
27. // m_logger->info("[CLK {}] EOC/SYNC ready for callback", m_clk);
28. } else {
29.
30. bool requires_reg_RW_mode = false;
31. if (req_it->type == Type::AIM) {
32. // m_logger->info("Checking AIM request: {}", req_it->str());
33. if (AiMISRInfo::opcode_requires_reg_RW_mod(req_it->opcode)) {
34. requires_reg_RW_mode = true;
35. }
36. }
37.
38. if (requires_reg_RW_mode ^ is_reg_RW_mode) {
39. req_it->command = m_dram->m_commands("TMOO");
40. is_reg_RW_mode = !is_reg_RW_mode;
41. }
42.
43. // If we find a real request to serve
44. m_logger->info("[CLK {}] Issuing {} for {}", m_clk, std::string(m_dram->m_commands(req_it->command)).c_str(), req_it->str());
45. if (req_it->issue == -1) {
46. req_it->issue = m_clk - 1;
47. }
48. m_dram->issue_command(req_it->command, req_it->addr_vec);
49. s_num_commands[req_it->command] += 1;
50. }
 }
```

```
1. void tick() override {
2. m_clk++;
3.
4. if (m_clk == m_next_refresh_cycle) {
5. m_next_refresh_cycle += m_nrefi;
6. for (int r = 0; r < m_num_ranks; r++) {
7. std::vector<int> addr_vec(m_dram_org_levels, -1);
8. addr_vec[0] = m_ctrl->m_channel_id;
9. addr_vec[1] = r;
10. Request req(addr_vec, m_ref_req_id);
11.
12. bool is_success = m_ctrl->priority_send(req);
13. if (!is_success) {
14. throw std::runtime_error("Failed to send refresh!");
15. }
16. }
17. }
18.};
```

### 1. Pending buffer에 있는 Request들을 완료 처리 & Refresh 관리 & Request Scheduling – 세부 사항 2 : m\_refresh->tick()

- Refreshing은 특정 주기마다 수행됨
- 이를 `m_clk == m_next_refresh_cycle`로 판단하여 특정 주기가 도래하면 Refresh Request를 priority buffer에 enqueue



# AiM Simulator MemoryController Function

## MemoryController: AiM\_dram\_controller.cpp – tick()

```
1. void tick() override {
2. m_clk++;
3. // if ((m_clk == 1) || (m_clk % 1000 == 0))
4. // m_logger->info("[CLK {}]", m_clk);
5.
6. // 1. Serve completed reads
7. serve_completed_reqs();
8.
9. m_refresh->tick();
10.
11. // 2. Try to find a request to serve.
12. ReqBuffer::iterator req_it;
13. ReqBuffer *buffer = nullptr;
14. bool request_found = schedule_request(req_it, buffer);
15.
16. // // 3. Update all plugins
17. // for (auto plugin : m_plugins) {
18. // plugin->update(request_found, req_it);
19. // }
20.
21. // 4. Finally, issue the commands to serve the request
22. if (request_found) {
23. if ((req_it->opcode == Opcode::ISR_EOC) || (req_it->opcode == Opcode::ISR_SYNC)) {
24. req_it->depart = m_clk;
25. pending_reads.push_back(*req_it);
26. buffer->remove(req_it);
27. // m_logger->info("[CLK {}] EOC/SYNC ready for callback", m_clk);
28. } else {
29.
30. bool requires_reg_RW_mode = false;
31. if (req_it->type == Type::AIM) {
32. // m_logger->info("Checking AiM request: {}", req_it->str());
33. if (AiMISIRInfo::opcode_requires_reg_RW_mod(req_it->opcode)) {
34. requires_reg_RW_mode = true;
35. }
36.
37. if (requires_reg_RW_mode ^ is_reg_RW_mode) {
38. req_it->command = m_dram->m_commands("TMOD");
39. is_reg_RW_mode = !is_reg_RW_mode;
40. }
41.
42. // If we find a real request to serve
43. m_logger->info("[CLK {}] Issuing {} for {}", m_clk, std::string(m_dram->m_commands(req_it->command)).c_str(), req_it->str());
44. if (req_it->issue == -1) {
45. req_it->issue = m_clk - 1;
46. }
47.
48. m_dram->issue_command(req_it->command, req_it->addr_vec);
49. s_num_commands[req_it->command] += 1;
50. }
51.
52. bool schedule_request(ReqBuffer::iterator &req_it, ReqBuffer *req_buffer) {
53. bool request_found = false;
54. // 2.1 First, check the act buffer to serve requests that are already activating (avoid useless ACTs)
55. if (req_it = m_scheduler->get_best_request(m_active_buffer); req_it != m_active_buffer.end()) {
56. if (m_dram->check_ready(req_it->command, req_it->addr_vec)) {
57. request_found = true;
58. req_buffer = &m_active_buffer;
59. // m_logger->info("[CLK {}] Found request in active buffer: {}", m_clk, req_it->str());
60. }
61. }
62.
63. // 2.2 If no requests can be scheduled from the act buffer, check the rest of the buffers
64. if (!request_found) {
65. // 2.2.1 We first check the priority buffer to prioritize e.g., maintenance requests
66. if (m_priority_buffer.size() != 0) {
67. req_buffer = &m_priority_buffer;
68. req_it = m_priority_buffer.begin();
69. req_it->command = m_dram->get_preq_command(req_it->final_command, req_it->addr_vec);
70. if ((request_found == false) & (m_priority_buffer.size() != 0)) {
71. request_found = true;
72. // m_logger->info("[CLK {}] Found request in priority buffer: {}", m_clk, req_it->str());
73. }
74. }
75.
76. // 2.2.1 If no request to be scheduled in the priority buffer, check the read and write OR AIM buffers.
77. if (!request_found) {
78. if (m_main_buffer.size() != 0) {
79. req_it = m_main_buffer.begin();
80. if ((req_it->opcode == Opcode::ISR_EOC) || (req_it->opcode == Opcode::ISR_SYNC)) {
81. req_buffer = &m_main_buffer;
82. // m_logger->info("[CLK {}] 1- Found AIM request in buffer: {}", m_clk, req_it->str());
83. return true;
84. }
85. req_it->command = m_dram->get_preq_command(req_it->final_command, req_it->addr_vec);
86. request_found = m_dram->check_ready(req_it->command, req_it->addr_vec);
87. req_buffer = &m_main_buffer;
88. // if (request_found) {
89. // m_logger->info("[CLK {}] 2- Found AIM request in buffer: {}", m_clk, req_it->str());
90. // }
91. }
92. }
93.
94. // Query the write policy to decide which buffer to serve
95. set_write_mode();
96. auto &buffer = m_is_write_mode ? m_write_buffer : m_read_buffer;
97. if (req_it = m_scheduler->get_best_request(buffer); req_it != buffer.end()) {
98. request_found = m_dram->check_ready(req_it->command, req_it->addr_vec);
99. req_buffer = &buffer;
100. }
101. if (request_found) {
102. // m_logger->info("[CLK {}] Found request in general buffers: {}", m_clk, req_it->str());
103. }
104. }
105. }
106. }
107. }
108. // 2.3 If we find a request to schedule, we need to check if it will close an opened row in the active buffer.
109. if (request_found) {
110. if (m_dram->m_command_meta(req_it->command).is_closing) {
111. std::vector<Addr_t> rowgroup((req_it->addr_vec).begin(), (req_it->addr_vec).begin() + m_row_addr_idx);
112.
113. // Search the active buffer with the row address (inkl. banks, etc.)
114. for (auto _it = m_active_buffer.begin(); _it != m_active_buffer.end(); _it++) {
115. std::vector<Addr_t> _it_rowgroup(_it->addr_vec.begin(), _it->addr_vec.begin() + m_row_addr_idx);
116. if (rowgroup == _it_rowgroup) {
117. // Invalidate this scheduling outcome if we are to interrupt a request in the active buffer
118. request_found = false;
119. }
120. }
121. }
122. }
123. return request_found;
124. }
```

### 1. Pending buffer에 있는 Request들을 완료 처리 & Refresh 관리 & Request Scheduling – 세부 사항 3 : schedule\_request()

- 1.3. 현재 Memory Controller는 Request Scheduling을 위하여 5개의 buffer를 사용 -> active / priority / aim / read / write buffer
- 1.3.1. active buffer는 이전에 ACT를 prerequisite command로 발행한 Read/Write Request를 관리 -> 우선순위: 1순위 (= 직전에 ACT를 발행한 Request 1순위로 scheduling)
- 1.3.2. priority buffer는 Refresh Request를 관리 -> 우선순위: 2순위 (= Refresh Request 2순위로 scheduling)
- 1.3.3. aim buffer ↔ read/write buffer는 상호독립적 (PPT page 20 참고)

read/write buffer: write buffer에 특정% 이상의 write request가 쌓여 있으면 write request 먼저 scheduling, 아닐 시 read request 먼저 scheduling

# AiM Simulator MemoryController Function

## MemoryController: AiM\_dram\_controller.cpp – tick()

```
1. void tick() override {
2. m_clk++;
3. // if ((m_clk == 1) || (m_clk % 1000 == 0))
4. // m_logger->info("[CLK {}]", m_clk);
5.
6. // 1. Serve completed reads
7. serve_completed_reqs();
8.
9. m_refresh->tick();
10.
11. // 2. Try to find a request to serve.
12. ReqBuffer::iterator req_it;
13. ReqBuffer *buffer = nullptr;
14. bool request_found = schedule_request(req_it, buffer);
15.
16. // // 3. Update all plugins
17. // for (auto plugin : m_plugins) {
18. // plugin->update(request_found, req_it);
19. // }
20.
21. // 4. Finally, issue the commands to serve the request
22. if (request_found) {
23. if ((req_it->opcode == Opcode::ISR_EOC) || (req_it->opcode == Opcode::ISR_SYNC)) {
24. req_it->depart = m_clk;
25. pending_reads.push_back("req_it");
26. buffer->remove(req_it);
27. // m_logger->info("[CLK {}] EOC/SYNC ready for callback", m_clk);
28. } else {
29.
30. bool requires_reg_RW_mode = false;
31. if (req_it->type == Type::AIM) {
32. // m_logger->info("Checking AiM request: {}", req_it->str());
33. if (AiMISIRInfo::opcode_requires_reg_RW_mod(req_it->opcode)) {
34. requires_reg_RW_mode = true;
35. }
36.
37. if (requires_reg_RW_mode ^ is_reg_RW_mode) {
38. req_it->command = m_dram->m_commands("TMOD");
39. is_reg_RW_mode = !is_reg_RW_mode;
40. }
41.
42. // If we find a real request to serve
43. m_logger->info("[CLK {}] Issuing {} for {}", m_clk, std::string(m_dram->m_commands(req_it->command)).c_str(), req_it->str());
44. if (req_it->issue == -1) {
45. req_it->issue = m_clk - 1;
46. }
47.
48. m_dram->issue_command(req_it->command, req_it->addr_vec);
49. s_num_commands[req_it->command] += 1;
50. }
51. }
52. }
53.
54. // 2.1. First, check the act buffer to serve requests that are already activating (avoid useless ACTs)
55. if (req_it = m_scheduler->get_best_request(m_active_buffer); req_it != m_active_buffer.end()) {
56. if (m_dram->check_ready(req_it->command, req_it->addr_vec)) {
57. request_found = true;
58. req_buffer = &m_active_buffer;
59. // m_logger->info("[CLK {}] Found request in active buffer: {}", m_clk, req_it->str());
60. }
61. }
62.
63. // 2.2. If no requests can be scheduled from the act buffer, check the rest of the buffers
64. if (!request_found) {
65. // 2.2.1. We first check the priority buffer to prioritize e.g., maintenance requests
66. if (m_priority_buffer.size() != 0) {
67. req_buffer = &m_priority_buffer;
68. req_it = m_priority_buffer.begin();
69. req_it->command = m_dram->get_req_command(req_it->final_command, req_it->addr_vec);
70. request_found = m_dram->check_ready(req_it->command, req_it->addr_vec);
71. if ((request_found == false) & (m_priority_buffer.size() != 0)) {
72. return false;
73. }
74. // if (request_found) {
75. // m_logger->info("[CLK {}] Found request in priority buffer: {}", m_clk, req_it->str());
76. // }
77. }
78.
79. // 2.2.1. If no request to be scheduled in the priority buffer, check the read and write OR AIM buffers.
80. if (!request_found) {
81. if (m_main_buffer.size() != 0) {
82. req_it = m_main_buffer.begin();
83. if ((req_it->opcode == Opcode::ISR_EOC) || (req_it->opcode == Opcode::ISR_SYNC)) {
84. req_buffer = &m_main_buffer;
85. // m_logger->info("[CLK {}] 1- Found AIM request in buffer: {}", m_clk, req_it->str());
86. return true;
87. }
88. req_it->command = m_dram->get_req_command(req_it->final_command, req_it->addr_vec);
89. request_found = m_dram->check_ready(req_it->command, req_it->addr_vec);
90. req_buffer = &m_main_buffer;
91. // if (request_found) {
92. // m_logger->info("[CLK {}] 2- Found AIM request in buffer: {}", m_clk, req_it->str());
93. // }
94. }
95. else {
96. // Query the write policy to decide which buffer to serve
97. set_write_mode();
98. auto &buffer = m_is_write_mode ? m_write_buffer : m_read_buffer;
99. if (req_it = m_scheduler->get_best_request(buffer); req_it != buffer.end()) {
100. request_found = m_dram->check_ready(req_it->command, req_it->addr_vec);
101. req_buffer = &buffer;
102. // if (request_found) {
103. // m_logger->info("[CLK {}] Found request in general buffers: {}", m_clk, req_it->str());
104. // }
105. }
106. }
107. }
108. }
109.
110. // 2.3 If we find a request to schedule, we need to check if it will close an opened row in the active buffer.
111. if (request_found) {
112. if (m_dram->m_command_meta(req_it->command).is_closing) {
113. std::vector<Addr_t> rowgroup((req_it->addr_vec).begin(), (req_it->addr_vec).begin() + m_row_addr_idx);
114.
115. // Search the active buffer with the row address (inkl. banks, etc.)
116. for (auto _it = m_active_buffer.begin(); _it != m_active_buffer.end(); _it++) {
117. std::vector<Addr_t> _it_rowgroup(_it->addr_vec.begin(), _it->addr_vec.begin() + m_row_addr_idx);
118. if (rowgroup == _it_rowgroup) {
119. // Invalidate this scheduling outcome if we are to interrupt a request in the active buffer
120. request_found = false;
121. }
122. }
123. }
124. }
125.
126. return request_found;
127. }
```

### 1. Pending buffer에 있는 Request들을 완료 처리 & Refresh 관리 & Request Scheduling – 세부 사항 3 : schedule\_request()

- schedule\_request()에서 사용되는 주요 함수 3가지를 알아본다 (get\_preq\_command() / check\_ready() / get\_best\_request())
- 1.3.4. get\_preq\_command(): 현재 Request가 접근하려는 DRAM의 주소를 확인하여 해당 주소를 접근하기 위해 필요한 prerequisite command를 가져옴  
(ex. Read Request를 Issue 시, 접근하려는 Row가 closed이면, Row를 Open하기 위해 ACT command를 먼저 Issue해야 함)
- 1.3.5. check\_ready(): 현재 Request가 접근하려는 DRAM의 주소에 도달하기 위한 timing constraints를 각 node별로 만족하는 지 확인 후, 만족 시 true를 반환  
(현재 code는 약간의 이상함이 존재하기에, 이에 대해서 주석을 달아 놓음 - 민성)
- 1.3.6. get\_best\_request(): 현재 buffer안에 있는 모든 Request에 대해서 get\_preq\_command(), check\_ready()를 실행하여, ready가 된 Request중 가장 먼저 들어온 Request를 반환

# AiM Simulator MemoryController Function

## MemoryController: AiM\_dram\_controller.cpp – tick()

```
1. void tick() override {
2. m_clk++;
3. // if ((m_clk == 1) || (m_clk % 1000 == 0))
4. // m_logger->info("[CLK {}]", m_clk);
5.
6. // 1. Serve completed reads
7. serve_completed_reqs();
8.
9. m_refresh->tick();
10.
11. // 2. Try to find a request to serve.
12. ReqBuffer::iterator req_it;
13. ReqBuffer *buffer = nullptr;
14. bool request_found = schedule_request(req_it, buffer);
15.
16. // 3. Update all plugins
17. // for (auto plugin : m_plugins) {
18. // plugin->update(request_found, req_it);
19. // }
20.
21. // 4. Finally, issue the commands to serve the request
22. if (request_found) {
23. if ((req_it->opcode == Opcode::ISR_EOC) || (req_it->opcode == Opcode::ISR_SYNC)) {
24. req_it->depart = m_clk;
25. pending_reads.push_back(*req_it);
26. buffer->remove(req_it);
27. // m_logger->info("[CLK {}] EOC/SYNC ready for callback", m_clk);
28. } else {
29. bool requires_reg_RW_mode = false;
30. if (req_it->type == Type::AIM) {
31. // m_logger->info("Checking AiM request: {}", req_it->str());
32. if (AiMISRInfo::opcode_requires_reg_RW_mod(req_it->opcode)) {
33. requires_reg_RW_mode = true;
34. }
35. }
36. }
37.
38. if (requires_reg_RW_mode ^ is_reg_RW_mode) {
39. req_it->command = m_dram->m_commands("TMOO");
40. is_reg_RW_mode = !is_reg_RW_mode;
41. }
42.
43. // If we find a real request to serve
44. m_logger->info("[CLK {}] Issuing {} for {}", m_clk, std::string(m_dram->m_commands(req_it->command)).c_str(), req_it->str());
45. if (req_it->issue == -1) {
46. req_it->issue = m_clk - 1;
47. }
48. m_dram->issue_command(req_it->command, req_it->addr_vec);
49. s_num_commands[req_it->command] += 1;
50. }
```

```
51. // If we are issuing the last command, set depart clock cycle and move the request to the pending_reads queue
52. if (req_it->command == req_it->final_command) {
53. int latency = m_dram->m_command_latencies(req_it->command);
54. assert(latency > 0);
55. req_it->depart = m_clk + latency;
56. if (req_it->is_reader()) {
57. pending_reads.push_back(*req_it);
58. } else {
59. pending_writes.push_back(*req_it);
60. }
61. if (req_it->type == Type::AIM) {
62. // - commented by MSKIM in ISL yonsei
63. // AiM_cycles update Logic 이 이상함
64. // 원래 code : m_clk - req_it->issue 의 경우 prerequisite command issue 시간만 포함됨
65. // 만약 scheduling buffer 대기시간 포함 & final_req의 latency 포함 : req_it->depart - req_it->arrive
66. // 만약 scheduling buffer 대기시간 포함 & final_req의 latency 미포함 : m_clk - req_it->arrive
67. // 만약 scheduling buffer 대기시간 미포함 & final_req의 latency 포함 : req_it->depart - req_it->issue
68. // 만약 scheduling buffer 대기시간 미포함 & final_req의 latency 미포함 : m_clk - req_it->issue
69. s_num_AiM_cycles[req_it->opcode] += (m_clk - req_it->issue);
70. } else {
71. // - commented by MSKIM in ISL yonsei
72. // RW_cycles update Logic 도 같은 이유로 이상함
73. s_num_RW_cycles[req_it->type] += (m_clk - req_it->issue);
74. }
75. // else if (req_it->type == Type::Write) {
76. // // TODO: Add code to update statistics
77. // }
78. buffer->remove(req_it);
79. if (req_it->type != Type::AIM) {
80. if (m_dram->m_command_meta(req_it->command).is_opening) {
81. m_active_buffer.enqueue(*req_it);
82. buffer->remove(req_it);
83. }
84. }
85. }
86. else if (m_read_buffer.size() == 0 && m_write_buffer.size() == 0 && m_aim_buffer.size() == 0 && pending_reads.size() == 0 && pending_writes.size() == 0) {
87. // if (m_channel_id == 0)
88. // m_logger->info("[CLK {}] CH0 IDLE", m_clk);
89. s_num_idle_cycles += 1;
90. }
91. // - added by MSKIM in ISL yonsei
92. else {
93. s_num_not_issued_cycles += 1;
94. }
95. // - - - - -
96. if (m_dram->m_open_rows[m_channel_id] == 0) {
97. s_num_precharged_cycles += 1;
98. } else {
99. s_num_active_cycles += 1;
100. }
101.
102. };
```

## 2. Scheduling된 Request가 존재 시 이를 issue 시킴 – ISR\_EOC 또는 ISR\_SYNC가 scheduling된 Request인 경우

- 2.1.1. ISR\_EOC: Simulation 종료 (= 모든 Request가 완료되고 ISR\_EOC가 callback되면 simulation 종료)
- 2.1.2. ISR\_SYNC: Memory System에서 ISR\_SYNC 이전의 Request가 전부 완료되기 전까지 추가적인 Request를 보내지 않음 (= Synchronize)

(모든 Request가 완료될 때까지 기다리는 것은 AiM\_DMA\_blocking을 통해 처리됨 – PPT page 17의 stalled\_AiM\_requests == 0 로직 사용)

# AiM Simulator MemoryController Function

## MemoryController: AiM\_dram\_controller.cpp – tick()

```
1. void tick() override {
2. m_clk++;
3. // if ((m_clk == 1) || (m_clk % 1000 == 0))
4. // m_logger->info("[CLK {}]", m_clk);
5.
6. // 1. Serve completed reads
7. serve_completed_reqs();
8.
9. m_refresh->tick();
10.
11. // 2. Try to find a request to serve.
12. ReqBuffer::iterator req_it;
13. ReqBuffer *buffer = nullptr;
14. bool request_found = schedule_request(req_it, buffer);
15.
16. // 3. Update all plugins
17. // for (auto plugin : m_plugins) {
18. // plugin->update(request_found, req_it);
19. // }
20.
21. // 4. Finally, issue the commands to serve the request
22. if (request_found) {
23. if ((req_it->opcode == Opcode::ISR_EOC) || (req_it->opcode == Opcode::ISR_SYNC)) {
24. req_it->depart = m_clk;
25. pending_reads.push_back(*req_it);
26. buffer->remove(req_it);
27. // m_logger->info("[CLK {}] EOC/SYNC ready for callback", m_clk);
28. } else {
29. bool requires_reg_RW_mode = false;
30. if (req_it->type == Type::AIM) {
31. // m_logger->info("Checking AiM request: {}", req_it->str());
32. if (AiMInfo::opcode_requires_reg_RW_mod(req_it->opcode)) {
33. requires_reg_RW_mode = true;
34. }
35. }
36. }
37.
38. if (requires_reg_RW_mode ^ is_reg_RW_mode) {
39. req_it->command = m_dram->m_commands("TMOD");
40. is_reg_RW_mode = !is_reg_RW_mode;
41. }
42.
43. // If we find a real request to serve
44. m_logger->info("[CLK {}] Issuing {} for {}", m_clk, std::string(m_dram->m_commands(req_it->command)).c_str(), req_it->str());
45. if (req_it->issue == -1) {
46. req_it->issue = m_clk - 1;
47. }
48. m_dram->issue_command(req_it->command, req_it->addr_vec);
49. s_num_commands[req_it->command] += 1;
50. }
```

```
51. // If we are issuing the last command, set depart clock cycle and move the request to the pending_reads queue
52. if (req_it->command == req_it->final_command) {
53. int latency = m_dram->m_command_latencies(req_it->command);
54. assert(latency > 0);
55. req_it->depart = m_clk + latency;
56. if (req_it->is_reader()) {
57. pending_reads.push_back(*req_it);
58. } else {
59. pending_writes.push_back(*req_it);
60. }
61. }
62. if (req_it->type == Type::AIM) {
63. // - commented by MSKIM in ISL yonsei
64. // AiM_cycles update Logic 이 이상함
65. // 원래 code : m_clk - req_it->issue 의 경우 prerequisite command issue 시간만 포함됨
66. // 만약 scheduling buffer 대기시간 포함 & final_req의 latency 포함 : req_it->depart - req_it->arrive
67. // 만약 scheduling buffer 대기시간 포함 & final_req의 latency 미포함 : m_clk - req_it->arrive
68. // 만약 scheduling buffer 대기시간 미포함 & final_req의 latency 포함 : req_it->depart - req_it->issue
69. // 만약 scheduling buffer 대기시간 미포함 & final_req의 latency 미포함 : m_clk - req_it->issue
70. s_num_AiM_cycles[req_it->opcode] += (m_clk - req_it->issue);
71. }
72. // - commented by MSKIM in ISL yonsei
73. // RW_cycles update Logic 도 같은 이유로 이상함
74. s_num_RW_cycles[req_it->type] += (m_clk - req_it->issue);
75.
76. // else if (req_it->type == Type::Write) {
77. // // TODO: Add code to update statistics
78. // }
79. buffer->remove(req_it);
80. // else if (req_it->type != Type::AIM) {
81. // if (m_dram->m_command_meta(req_it->command).is_opening) {
82. // m_active_buffer.enqueue(*req_it);
83. // buffer->remove(req_it);
84. // }
85. // }
86. // else if (m_read_buffer.size()==0 && m_write_buffer.size()==0 && m_aim_buffer.size()==0 && pending_reads.size()==0 && pending_writes.size()==0) {
87. // // if (m_channel_id == 0)
88. // // m_logger->info("[CLK {}] CH0 IDLE", m_clk);
89. // s_num_idle_cycles += 1;
90. // }
91. // - added by MSKIM in ISL yonsei
92. // else {
93. // s_num_not_issued_cycles += 1;
94. // }
95. // - added by MSKIM in ISL yonsei
96. // if (m_dram->m_open_rows[m_channel_id] == 0) {
97. // s_num_precharged_cycles += 1;
98. // }
99. // else {
100. // s_num_active_cycles += 1;
101. // }
102. };
```

## 2. Scheduling된 Request가 존재 시 이를 issue 시킴 – ISR\_EOC 또는 ISR\_SYNC가 scheduling된 Request가 아닌 경우

- 2.1.3. Type::AIM인 Request 중 requires\_reg\_RW\_mode = true가 필요한 Request의 경우 requires\_reg\_RW\_mode = true로 변경

현재 DRAM이 is\_reg\_RW\_mode = true가 아니면 TMOD command를 scheduling하여 is\_reg\_RW\_mod를 변경

(AiM은 Global Buffer / MAC accumulator regs / Activation Function regs를 접근 시 is\_reg\_RW\_mod = true여야 함 (ex. ISR\_WR\_GB, ISR\_RD\_MAC, ISR\_RD\_AF 명령어 등))

# AiM Simulator MemoryController Function

## MemoryController: AiM\_dram\_controller.cpp – tick()

```
1. void tick() override {
2. m_clk++;
3. // if ((m_clk == 1) || (m_clk % 1000 == 0))
4. // m_logger->info("[CLK {}]", m_clk);
5.
6. // 1. Serve completed reads
7. serve_completed_reqs();
8.
9. m_refresh->tick();
10.
11. // 2. Try to find a request to serve.
12. ReqBuffer::iterator req_it;
13. ReqBuffer *buffer = nullptr;
14. bool request_found = schedule_request(req_it, buffer);
15.
16. // 3. Update all plugins
17. // for (auto plugin : m_plugins) {
18. // plugin->update(request_found, req_it);
19. // }
20.
21. // 4. Finally, issue the commands to serve the request
22. if (request_found) {
23. if ((req_it->opcode == Opcode::ISR_EOC) || (req_it->opcode == Opcode::ISR_SYNC)) {
24. req_it->depart = m_clk;
25. pending_reads.push_back(*req_it);
26. buffer->remove(req_it);
27. // m_logger->info("[CLK {}] EOC/SYNC ready for callback", m_clk);
28. } else {
29. bool requires_reg_rw_mode = false;
30. if (req_it->type == Type::AIM) {
31. // m_logger->info("Checking AiM request: {}", req_it->str());
32. if (AiMInfo::opcode_requires_reg_rw_mod(req_it->opcode)) {
33. requires_reg_rw_mode = true;
34. }
35. }
36. }
37.
38. if (requires_reg_rw_mode ^ is_reg_rw_mode) {
39. req_it->command = m_dram->m_commands("TMOO");
40. is_reg_rw_mode = !is_reg_rw_mode;
41. }
42.
43. // If we find a real request to serve
44. m_logger->info("[CLK {}] Issuing {} for {}", m_clk, std::string(m_dram->m_commands(req_it->command)).c_str(), req_it->str());
45. if (req_it->issue == -1) {
46. req_it->issue = m_clk - 1;
47. }
48. m_dram->issue_command(req_it->command, req_it->addr_vec);
49. s_num_commands[req_it->command] += 1;
50. }
```

```
51. // If we are issuing the last command, set depart clock cycle and move the request to the pending_reads queue
52. if (req_it->command == req_it->final_command) {
53. int latency = m_dram->m_command_latencies(req_it->command);
54. assert(latency > 0);
55. req_it->depart = m_clk + latency;
56. if (req_it->is_reader()) {
57. pending_reads.push_back(*req_it);
58. } else {
59. pending_writes.push_back(*req_it);
60. }
61. }
62. if (req_it->type == Type::AIM) {
63. // - commented by MSKIM in ISL yonsei
64. // AiM_cycles update Logic 이 이상함
65. // 원래 code : m_clk - req_it->issue 의 경우 prerequisite command issue 시간만 포함됨
66. // 만약 scheduling buffer 대기시간 포함 & final_req의 latency 포함 : req_it->depart - req_it->arrive
67. // 만약 scheduling buffer 대기시간 포함 & final_req의 latency 미포함 : m_clk - req_it->arrive
68. // 만약 scheduling buffer 대기시간 미포함 & final_req의 latency 포함 : req_it->depart - req_it->issue
69. // 만약 scheduling buffer 대기시간 미포함 & final_req의 latency 미포함 : m_clk - req_it->issue
70. s_num_AiM_cycles[req_it->opcode] += (m_clk - req_it->issue);
71. } else {
72. // - commented by MSKIM in ISL yonsei
73. // RW_cycles update Logic 도 같은 이유로 이상함
74. s_num_RW_cycles[req_it->type] += (m_clk - req_it->issue);
75. }
76. // else if (req_it->type == Type::Write) {
77. // // TODO: Add code to update statistics
78. // }
79. buffer->remove(req_it);
80. if (req_it->type != Type::AIM) {
81. if (m_dram->m_command_meta(req_it->command).is_opening) {
82. m_active_buffer.enqueue(*req_it);
83. buffer->remove(req_it);
84. }
85. }
86. } else if (m_read_buffer.size() == 0 && m_write_buffer.size() == 0 && m_aim_buffer.size() == 0 && pending_reads.size() == 0 && pending_writes.size() == 0) {
87. // if (m_channel_id == 0)
88. // m_logger->info("[CLK {}] CH0 IDLE", m_clk);
89. s_num_idle_cycles += 1;
90. }
91. // - added by MSKIM in ISL yonsei
92. else {
93. s_num_not_issued_cycles += 1;
94. }
95. // - added by MSKIM in ISL yonsei
96. if (m_dram->m_open_rows[m_channel_id] == 0) {
97. s_num_precharged_cycles += 1;
98. } else {
99. s_num_active_cycles += 1;
100. }
101. }
102. };
```

## 2. Scheduling된 Request가 존재 시 이를 issue 시킴 – ISR\_EOC 또는 ISR\_SYNC가 scheduling된 Request가 아닌 경우

- 2.1.4. 만약 해당 Request가 처음 issue되는 것이면 issue된 clk을 기록함  
(ex. Read Request는 ACT -> RD 순으로 command가 진행되므로, ACT issue 시 issue된 clk 기록 [Row가 closed라는 가정 하에])
- 2.1.5. DRAM에 해당 Command를 issue하라고 명령함



# AiM Simulator MemoryController Function

## MemoryController: AiM\_dram\_controller.cpp – tick()

```
1. void tick() override {
2. m_clk++;
3. // if ((m_clk == 1) || (m_clk % 1000 == 0))
4. // m_logger->info("[CLK {}]", m_clk);
5.
6. // 1. Serve completed reads
7. serve_completed_reqs();
8.
9. m_refresh->tick();
10.
11. // 2. Try to find a request to serve.
12. ReqBuffer::iterator req_it;
13. ReqBuffer *buffer = nullptr;
14. bool request_found = schedule_request(req_it, buffer);
15.
16. // 3. Update all plugins
17. // for (auto plugin : m_plugins) {
18. // plugin->update(request_found, req_it);
19. // }
20.
21. // 4. Finally, issue the commands to serve the request
22. if (request_found) {
23. if ((req_it->opcode == Opcode::ISR_EOC) || (req_it->opcode == Opcode::ISR_SYNC)) {
24. req_it->depart = m_clk;
25. pending_reads.push_back(*req_it);
26. buffer->remove(req_it);
27. // m_logger->info("[CLK {}] EOC/SYNC ready for callback", m_clk);
28. } else {
29. bool requires_reg_RW_mode = false;
30. if (req_it->type == Type::AIM) {
31. // m_logger->info("Checking AiM request: {}", req_it->str());
32. if (AiMISRInfo::opcode_requires_reg_RW_mod(req_it->opcode)) {
33. requires_reg_RW_mode = true;
34. }
35. }
36. }
37.
38. if (requires_reg_RW_mode ^ is_reg_RW_mode) {
39. req_it->command = m_dram->m_commands("TMOO");
40. is_reg_RW_mode = !is_reg_RW_mode;
41. }
42.
43. // If we find a real request to serve
44. m_logger->info("[CLK {}] Issuing {} for {}", m_clk, std::string(m_dram->m_commands(req_it->command)).c_str(), req_it->str());
45. if (req_it->issue == -1) {
46. req_it->issue = m_clk - 1;
47. }
48. m_dram->issue_command(req_it->command, req_it->addr_vec);
49. s_num_commands[req_it->command] += 1;
50. }
```

```
51. // If we are issuing the last command, set depart clock cycle and move the request to the pending_reads queue
52. if (req_it->command == req_it->final_command) {
53. int latency = m_dram->m_command_latencies(req_it->command);
54. assert(latency > 0);
55. req_it->depart = m_clk + latency;
56. if (req_it->is_reader()) {
57. pending_reads.push_back(*req_it);
58. } else {
59. pending_writes.push_back(*req_it);
60. }
61. if (req_it->type == Type::AIM) {
62. // - commented by MSKIM in ISL yonsei
63. // AiM_cycles update Logic 이 이상함
64. // 원래 code : m_clk - req_it->issue 의 경우 prerequisite command issue 시간만 포함됨
65. // 만약 scheduling buffer 대기시간 포함 & final_req의 latency 포함 : req_it->depart - req_it->arrive
66. // 만약 scheduling buffer 대기시간 포함 & final_req의 latency 미포함 : m_clk - req_it->arrive
67. // 만약 scheduling buffer 대기시간 미포함 & final_req의 latency 포함 : req_it->depart - req_it->issue
68. // 만약 scheduling buffer 대기시간 미포함 & final_req의 latency 미포함 : m_clk - req_it->issue
69. s_num_AiM_cycles[req_it->opcode] += (m_clk - req_it->issue);
70. } else {
71. // - commented by MSKIM in ISL yonsei
72. // RW_cycles update Logic 도 같은 이유로 이상함
73. s_num_RW_cycles[req_it->type] += (m_clk - req_it->issue);
74. }
75. // else if (req_it->type == Type::Write) {
76. // // TODO: Add code to update statistics
77. // }
78. buffer->remove(req_it);
79. if (req_it->type != Type::AIM) {
80. if (m_dram->m_command_meta(req_it->command).is_opening) {
81. m_active_buffer.enqueue(*req_it);
82. buffer->remove(req_it);
83. }
84. }
85. }
86. else if (m_read_buffer.size() == 0 && m_write_buffer.size() == 0 && m_aim_buffer.size() == 0 && pending_reads.size() == 0 && pending_writes.size() == 0) {
87. // if (m_channel_id == 0)
88. // m_logger->info("[CLK {}] CH0 IDLE", m_clk);
89. s_num_idle_cycles += 1;
90. }
91. // - added by MSKIM in ISL yonsei
92. else {
93. s_num_not_issued_cycles += 1;
94. }
95. // - added by MSKIM in ISL yonsei
96. if (m_dram->m_open_rows[m_channel_id] == 0) {
97. s_num_precharged_cycles += 1;
98. } else {
99. s_num_active_cycles += 1;
100. }
101. }
102. };
```

### 3. Issue 시킨 Request에 대해 statistics를 update

- 3.1. 현재 issue한 command가 해당 Request의 final command였으면 pending queue에 enqueue함
- 3.2. 해당 Request가 처음 issue된 clk부터 final command가 issue된 clk까지의 cycles를 statistic에 update
- 3.3. 그 외의 cycles update (idle\_cycles: buffer가 다 비어있던 cycle 수 / precharged\_cycles : 해당 channel의 모든 Banks가 row\_closed 시 increment / active\_cycles : 해당 channel의 모든 Banks 중 1개의 Bank라도 row가 open되어 있으면 increment)



# AiM Simulator DRAM issue\_command Function

## ❑ Memory: GDDR6.cpp – issue\_command()

```
1. void issue_command(int command, const AddrVec_t &addr_vec) override {
2. int channel_id = addr_vec[m_levels["channel"]];
3. m_channels[channel_id] -> update_timing(command, addr_vec, m_clk);
4. m_channels[channel_id] -> update_states(command, addr_vec, m_clk);
5. switch (command) {
6. case m_commands["WRA16"]:
7. case m_commands["PREA"]: {
8. m_open_rows[channel_id] = 0;
9. break;
10. }
11. case m_commands["PRE4"]: {
12. int bankgroup_id = addr_vec[m_levels["bankgroup"]];
13. for (int bank_id = bankgroup_id * 4; bank_id < (bankgroup_id + 1) * 4; bank_id++) {
14. m_open_rows[channel_id] &= ~(uint16_t)(1 << bank_id);
15. }
16. break;
17. }
18. case m_commands["PRE"]:
19. case m_commands["RDA"]:
20. case m_commands["WRA"]: {
21. int bank_id = addr_vec[m_levels["bank"]];
22. m_open_rows[channel_id] &= ~(uint16_t)(1 << bank_id);
23. break;
24. }
25. case m_commands["ACT16"]: {
26. m_open_rows[channel_id] = (uint16_t)(0xFFFF);
27. break;
28. }
29. case m_commands["ACT4"]: {
30. int bankgroup_id = addr_vec[m_levels["bankgroup"]];
31. for (int bank_id = bankgroup_id * 4; bank_id < (bankgroup_id + 1) * 4; bank_id++) {
32. m_open_rows[channel_id] |= (uint16_t)(1 << bank_id);
33. }
34. break;
35. }
36. case m_commands["ACT"]: {
37. int bank_id = addr_vec[m_levels["bank"]];
38. m_open_rows[channel_id] |= (uint16_t)(1 << bank_id);
39. break;
40. }
41. }
42. }
```

```
1. void update_states(int command, const AddrVec_t &addr_vec, Clk_t clk) {
2. if (m_spec->m_actions[m_level][command]) {
3. // update the state machine at this level
4. m_spec->m_actions[m_level][command](static_cast<NodeType *>(this), command, addr_vec, clk);
5. }
6. if (m_level == m_spec->m_command_scopes[command] || !m_child_nodes.size()) {
7. // stop recursion: updated all levels
8. return;
9. }
10. // recursively update child nodes
11. int child_id = addr_vec[m_level + 1];
12. if (child_id < 0) {
13. // stop recursion: there were no prerequisites at any level
14. return;
15. }
16. assert(child_id < m_child_nodes.size());
17. m_child_nodes[child_id] -> update_states(command, addr_vec, clk);
18. }
```

```
1. void set_actions() {
2. m_actions.resize(m_levels.size(), std::vector<ActionFunc_t<Node>>(m_commands.size()));
3.
4. // Channel Actions
5. m_actions[m_levels["channel"]][m_commands["WRA16"]] = Lambdas::Action::Channel::PREab<GDDR6>;
6. m_actions[m_levels["channel"]][m_commands["PREA"]] = Lambdas::Action::Channel::PREab<GDDR6>;
7. m_actions[m_levels["channel"]][m_commands["ACT16"]] = Lambdas::Action::Channel::ACTab<GDDR6>;
8.
9. // Bank Group Actions
10. m_actions[m_levels["bankgroup"]][m_commands["PRE4"]] = Lambdas::Action::BankGroup::PRE4b<GDDR6>;
11. m_actions[m_levels["bankgroup"]][m_commands["ACT4"]] = Lambdas::Action::BankGroup::ACT4b<GDDR6>;
12.
13. // Bank Actions
14. m_actions[m_levels["bank"]][m_commands["ACT"]] = Lambdas::Action::Bank::ACT<GDDR6>;
15. m_actions[m_levels["bank"]][m_commands["PRE"]] = Lambdas::Action::Bank::PRE<GDDR6>;
16. m_actions[m_levels["bank"]][m_commands["RDA"]] = Lambdas::Action::Bank::PRE<GDDR6>;
17. m_actions[m_levels["bank"]][m_commands["WRA"]] = Lambdas::Action::Bank::PRE<GDDR6>;
18. }
```

```
1. namespace Action {
2. namespace Bank {
3. template <class T>
4. void ACT(typename T::Node *node, int cmd, const AddrVec_t &addr_vec, Clk_t clk) {
5. int target_id = addr_vec[node->m_level + 1];
6. assert(node->m_state == T::m_states["Closed"]);
7. assert(node->m_row_state.size() == 0);
8. node->m_state = T::m_states["Opened"];
9. node->m_row_state[target_id] = T::m_states["Opened"];
10. };
11.
12. template <class T>
13. void PRE(typename T::Node *node, int cmd, const AddrVec_t &addr_vec, Clk_t clk) {
14. node->m_state = T::m_states["Closed"];
15. }
```

- 1. update\_timings()
  - GDDR6.cpp에 정의해 놓은 각 명령어들에 따른 timing constraints를 참조하여 각 level(= channel, ..., row, column)에서의 각 명령어가 발행될 수 있는 timing을 update
- 2. update\_states()
  - m\_actions()를 통해서 각 명령어가 DRAM의 state를 어떻게 바꾸는지 정의한 함수를 호출 (ex. ACT이면 row open으로 DRAM의 특정 Bank의 state를 바꿈)
- 3. 그 외 case문
  - AiM DRAM Controller에서의 statistic 중 s\_num\_precharged\_cycles라는 statistic을 update시 사용하는 m\_open\_row[]배열을 관리하여, 어떤 channel의 어떤 Bank가 row\_opened인지 추적

# Appendix – AiM\_GDDR6 commands & constraints

## ❑ MemoryController: GDDR6.cpp

```
1. inline static const ImplLUT m_command_scopes = LUT(
2. m_commands, m_levels, {
3. {"REFab", "channel"},
4. {"REFpb", "bank"},
5. {"ACT16", "channel"},
6. {"ACT4", "bankgroup"},
7. {"ACT", "row"},
8. {"PREA", "channel"},
9. {"PRE4", "bankgroup"},
10. {"PRE", "bank"},
11. {"RD", "column"},
12. {"WR", "column"},
13. {"RDA", "column"},
14. {"WRA", "column"},
15. {"MAC", "column"},
16. {"MAC16", "channel"},
17. {"AF16", "channel"},
18. {"EWMUL16", "channel"},
19. {"RDCP", "column"},
20. {"WRCP", "column"},
21. {"WRGB", "channel"},
22. {"RDMAC16", "channel"},
23. {"RDAF16", "channel"},
24. {"WRMAC16", "channel"},
25. {"WRA16", "channel"},
26. {"TMOD", "channel"},
27. {"SYNC", "channel"},
28. {"EOC", "channel"},
29. });
```

```
1. inline static constexpr ImplDef m_commands = {
2. //figure 3
3. "ACT",
4. "PREA",
5. "PRE",
6. "RD",
7. "WR",
8. "RDA",
9. "WRA",
10. "REFab",
11. "REFpb",
12. "ACT4",
13. "ACT16",
14. "PRE4",
15. "MAC",
16. "MAC16",
17. "AF16",
18. "EWMUL16",
19. "RDCP",
20. "WRCP",
21. "WRGB",
22. "RDMAC16",
23. "RDAF16",
24. "WRMAC16",
25. "WRA16",
26. "TMOD",
27. "SYNC",
28. "EOC",
29. "UNKNOWN";
```

```
1. inline static const ImplLUT m_command_meta = LUT<DRAMCommandMeta>(
2. m_commands, {
3. // open? close? access? refresh?
4. {"ACT", {true, false, false, false}},
5. {"PREA", {false, true, false, false}},
6. {"PRE", {false, true, false, false}},
7. {"RD", {false, false, true, false}},
8. {"WR", {false, false, true, false}},
9. {"RDA", {false, true, true, false}},
10. {"WRA", {false, true, true, false}},
11. {"REFab", {false, false, false, true}}, //double check
12. {"REFpb", {false, false, false, true}},
13. {"ACT4", {true, false, false, false}},
14. {"ACT16", {true, false, false, false}},
15. {"PRE4", {false, true, false, false}},
16. {"MAC", {false, false, true, false}},
17. {"MAC16", {false, false, true, false}},
18. {"AF16", {false, false, false, false}},
19. {"EWMUL16", {false, false, true, false}},
20. {"RDCP", {false, false, true, false}},
21. {"WRCP", {false, false, true, false}},
22. {"WRGB", {false, false, false, false}},
23. {"RDMAC16", {false, false, false, false}},
24. {"RDAF16", {false, false, false, false}},
25. {"WRMAC16", {false, false, false, false}},
26. {"WRA16", {false, true, true, false}},
27. {"TMOD", {false, false, false, false}},
28. {"SYNC", {false, false, false, false}},
29. {"EOC", {false, false, false, false}},
30. });
```

# Appendix – AiM\_GDDR6 commands & constraints

## ❑ MemoryController: GDDR6.cpp

```
1. inline static constexpr ImplDef m_requests = {
2. "read",
3. "write",
4. "all-bank-refresh",
5. "PREsb");
6.
7. inline static constexpr ImplDef m_aim_requests = {
8. "MIN",
9. "ISR_WR_SBK",
10. "ISR_WR_GB",
11. "ISR_WR_BIAS",
12. "ISR_WR_AFLUT",
13. "ISR_RD_MAC",
14. "ISR_RD_AF",
15. "ISR_RD_SBK",
16. "ISR_COPY_BKGB",
17. "ISR_COPY_GBBK",
18. "ISR_MAC_SBK",
19. "ISR_MAC_ABK",
20. "ISR_AF",
21. "ISR_EWMUL",
22. "ISR_EWADD",
23. "ISR_WR_ABK",
24. "ISR_SYNC",
25. "ISR_EOC",
26. "MAX"};
```

```
1. inline static const ImplLUT m_request_translations = LUT(
2. m_requests, m_commands, {
3. {"read", "RD"}, // Read single bank
4. {"write", "WR"}, // Write single bank
5. {"all-bank-refresh", "REFab"}, // Referesh all banks
6. {"PREsb", "PRE"}, // Referesh single bank
7. });
8.
9. inline static const ImplLUT m_aim_request_translations = LUT(
10. m_aim_requests, m_commands, {
11. {"MIN", "UNKNOWN"}, // 0 - Unknown and illegal
12. {"ISR_WR_SBK", "WR"}, // 1 - Write single bank
13. {"ISR_WR_GB", "WRGB"}, // 2 - Write global buffer
14. {"ISR_WR_BIAS", "WRMAC16"}, // 3 - Write all MAC registers
15. {"ISR_WR_AFLUT", "UNKNOWN"}, // 4 - Unknown and illegal
16. {"ISR_RD_MAC", "RDMAC16"}, // 5 - Read all MAC registers
17. {"ISR_RD_AF", "RDAF16"}, // 6 - Read all AF16 registers
18. {"ISR_RD_SBK", "RD"}, // 7 - Read single bank
19. {"ISR_COPY_BKGB", "RDCP"}, // 8 - Copy from a bank to the global buffer
20. {"ISR_COPY_GBBK", "WRCP"}, // 9 - Copy from the global buffer to a bank
21. {"ISR_MAC_SBK", "MAC"}, // 10 - MAC single bank
22. {"ISR_MAC_ABK", "MAC16"}, // 11 - MAC all bank
23. {"ISR_AF", "AF16"}, // 12 - AF16 all banks
24. {"ISR_EWMUL", "EWMUL16"}, // 14 - EWMUL16 all banks or 1 bank group
25. {"ISR_EWADD", "UNKNOWN"}, // 15 - Unknown and illegal
26. {"ISR_WR_ABK", "WRA16"}, // 1 - Write single bank
27. {"ISR_SYNC", "SYNC"}, // 16 - Unknown and illegal
28. {"ISR_EOC", "EOC"}, // 16 - Unknown and illegal
29. {"MAX", "UNKNOWN"}, // 17 - Unknown and illegal
30. });
```

```
1. inline static constexpr ImplDef m_states = {
2. "Opened", "Closed", "PowerUp", "N/A"};
3.
4. inline static const ImplLUT m_init_states = LUT(
5. m_levels, m_states, {
6. {"channel", "N/A"},
7. {"bankgroup", "N/A"},
8. {"bank", "Closed"},
9. {"row", "Closed"},
10. {"column", "N/A"},
11. });
12.
13.
```

# Appendix – AiM\_GDDR6 commands & constraints

## ❑ MemoryController: GDDR6.cpp

```
1. inline static constexpr ImplDef m_timings = {
2. "rate",
3. "nBL",
4. "nCL",
5. "nRCDRD",
6. "nRCDRDMAC",
7. "nRCDEWMUL",
8. "nRCDRDAF",
9. "nRCDRDCP",
10. "nRCDWR",
11. "nRCDWRCP",
12. "nRP",
13. "nRAS",
14. "nRC",
15. "nWR",
16. "nRTP",
17. "nCWL",
18. "nCCDS",
19. "nCCDL",
20. "nRRDS",
21. "nRRDL",
22. "nWTRS",
23. "nWTRL",
24. "nFAW",
25. "nRFC",
26. "nREFI",
27. "nRREFD",
28. "nRFCpb",
29. "nCLREG",
30. "nCLGB",
31. "nCWLREG",
32. "nCWLGB",
33. "nWPRE",
34. "nMODCH",
35. "tCK_ps"};
```

```
1. {"GDDR6_AiM_timing", // name
2. {
3. 2000, // rate
4. 2, // nBL (changed 1.25)
5. 50, // nCL (changed)
6. 36, // nRCDRD (changed)
7. 56, // nRCDRDMAC (changed + 10ns offset)
8. 25, // nRCDEWMUL (changed, calculated based on SDK)
9. 86, // nRCDRDAF (changed + 25ns offset)
10. 66, // nRCDRDCP (changed + 15ns offset)
11. 28, // nRCDWR (changed)
12. 48, // nRCDWRCP (changed + 10ns)
13. 32, // nRP (changed)
14. 54, // nRAS (changed from SIM: RDSBK (opsize=1) = ACT + RD + PRE = nRAS + nRP = nRAS + 32 = 86)
15. 89, // nRC (not imported, used only for precharge. TODO: It's important. It's also the ACT->ACT latency)
16. 33, // nWR (changed 16.625)
17. 12, // nRTP (changed 6.25)
18. 6, // nCWL
19. 2, // nCCDS (changed from ISSCC)
20. 2, // nCCDL (based on Xavier's WRSBK and RDSBK profiling results)
21. 11, // nRRDS
22. 11, // nRRDL
23. 9, // nWTRS
24. 11, // nWTRL
25. 42, // nFAW (not used)
26. 210, // nRFC
27. 105, // nRFCpb
28. 21, // nRREFD
29. 3333, // nREFI
30. 0, // nCLREG
31. 1, // nCLGB
32. 1, // nCWLREG
33. 1, // nCWLGB
34. 1, // nWPRE
35. 32, // nMODCH
36. 500 // tCK_ps
37. }
38. }
```